

HOL, Universal Meta-Logical Reasoning and LogiKEy

Christoph Benz Müller

U Bamberg (& FU Berlin)

*“If we had it [*a characteristica universalis*], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”* (Leibniz, 1690; translation by Russell)

AuReLeE Workshop, U Luxembourg, July 2022

Presentation of a new (interesting) result — AuReLeE work on this WE!

Outline

- Motivation
- **HOL** — Classical Higher-Order Logic
 - Non-Classical Logics (why Classical Logic is not enough)
- **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy** — Flexible KR&R Methodology
- Applications:
 - Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

Outline

- Motivation
- **HOL** — Classical Higher-Order Logic
 - Non-Classical Logics (why Classical Logic is not enough)
- **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy** — Flexible KR&R Methodology
- Applications:
 - Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

Research Interests

Since three decades:

Artificial Intelligence

Since more than two decades

- Automation of HOL: Leo Provers & TPTP Infrastructure
- Proof Assistants: Interaction & Automation
- Traditional Applications: Formal Methods in Maths, CS, AI

More recently

- Universal (Meta-)Logical Reasoning in HOL
- Novel Applications
 - Computational Philosophy
 - Foundations (Metaphysics, Maths, NL)
 - Ethics & Law

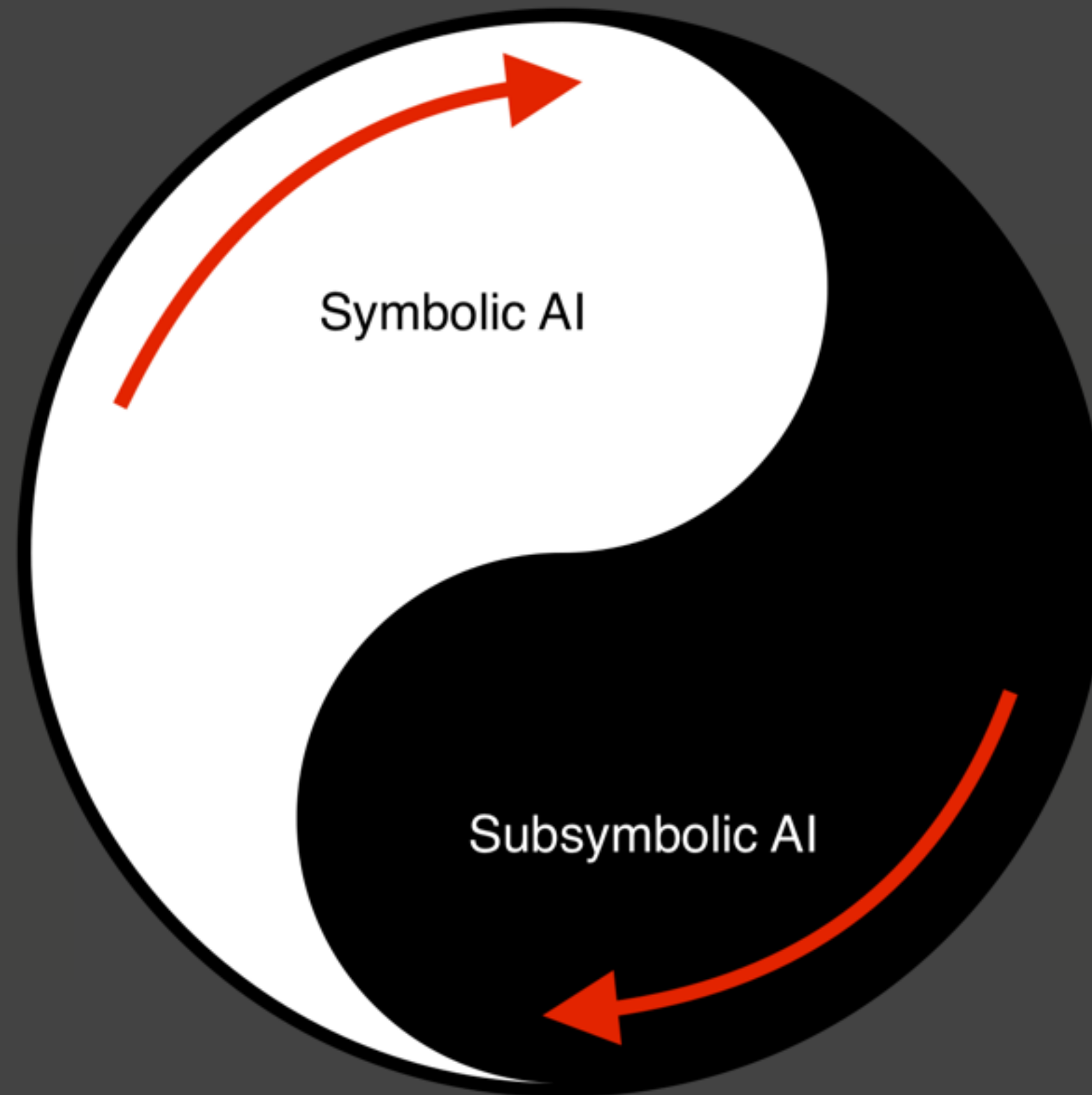
Artificial Intelligence

symbolic

Mind
Reasoning
Deductive
Little Data
Causalities
Precise

subsymbolic

Brain
Learning
Inductive
Massive Data
Correlations
Robust



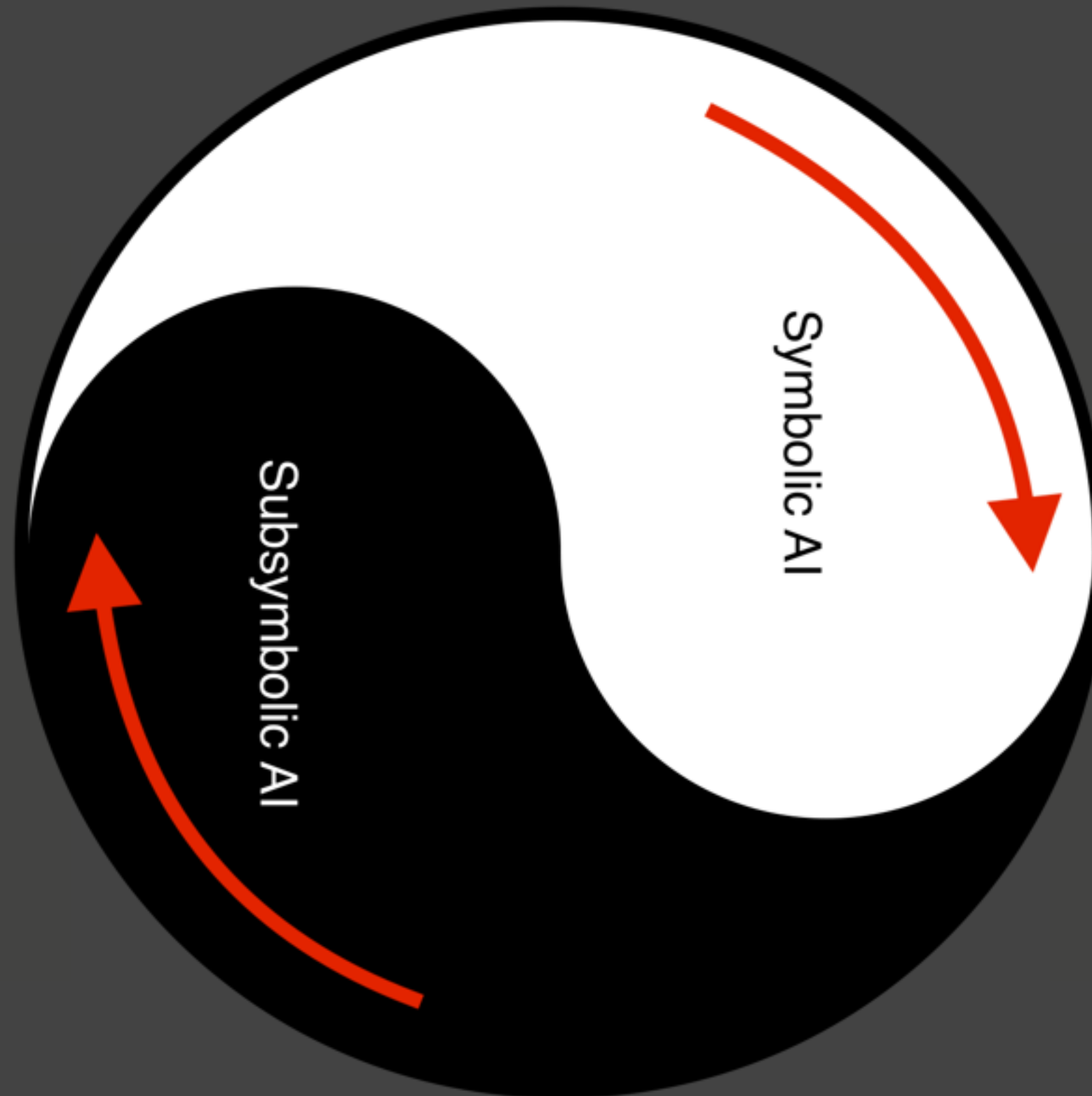
Artificial Intelligence

symbolic

Mind
Reasoning
Deductive
Little Data
Causalities
Precise

subsymbolic

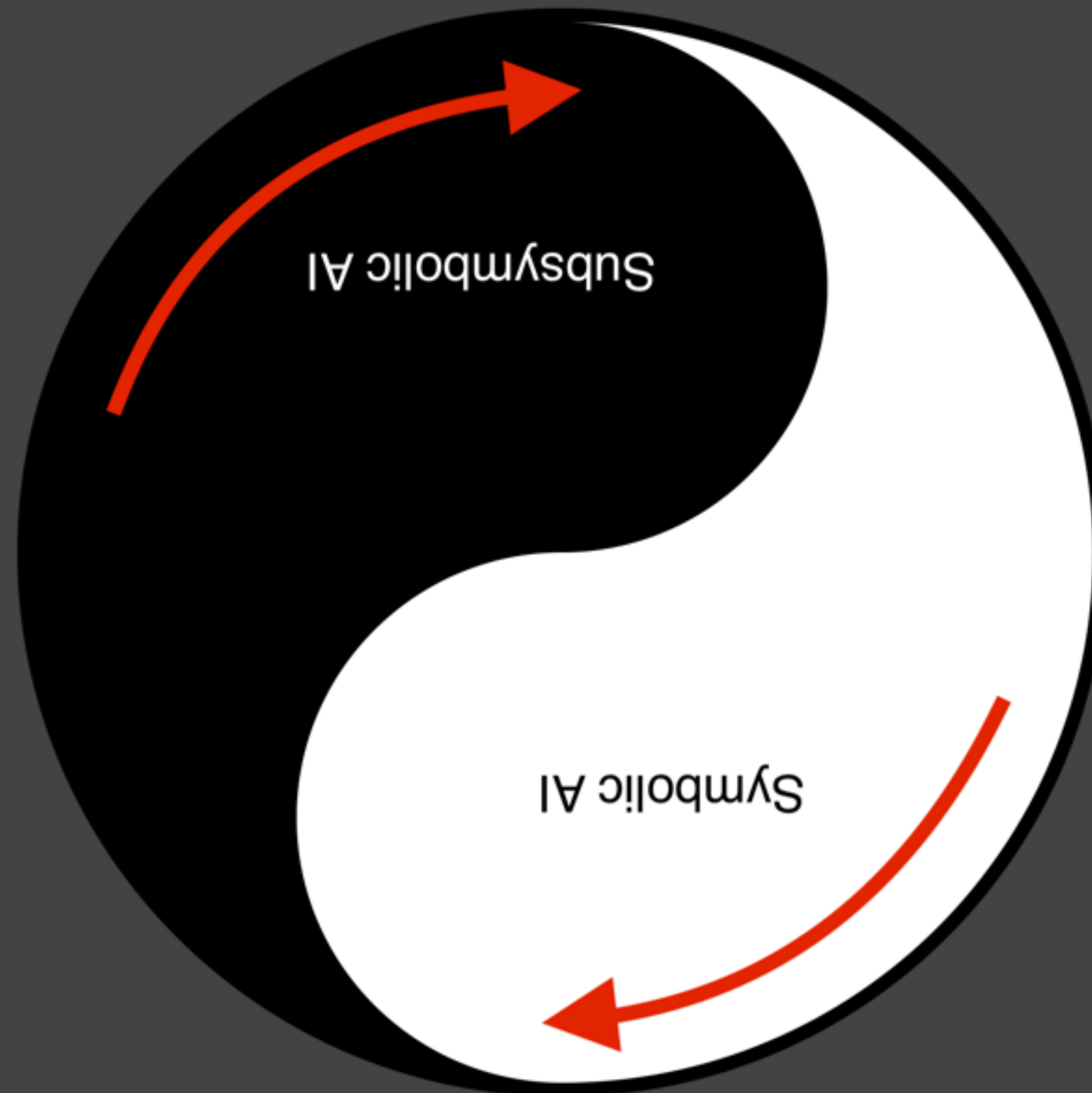
Brain
Learning
Inductive
Massive Data
Correlations
Robust



Artificial Intelligence

symbolic

Mind
Reasoning
Deductive
Little Data
Causalities
Precise



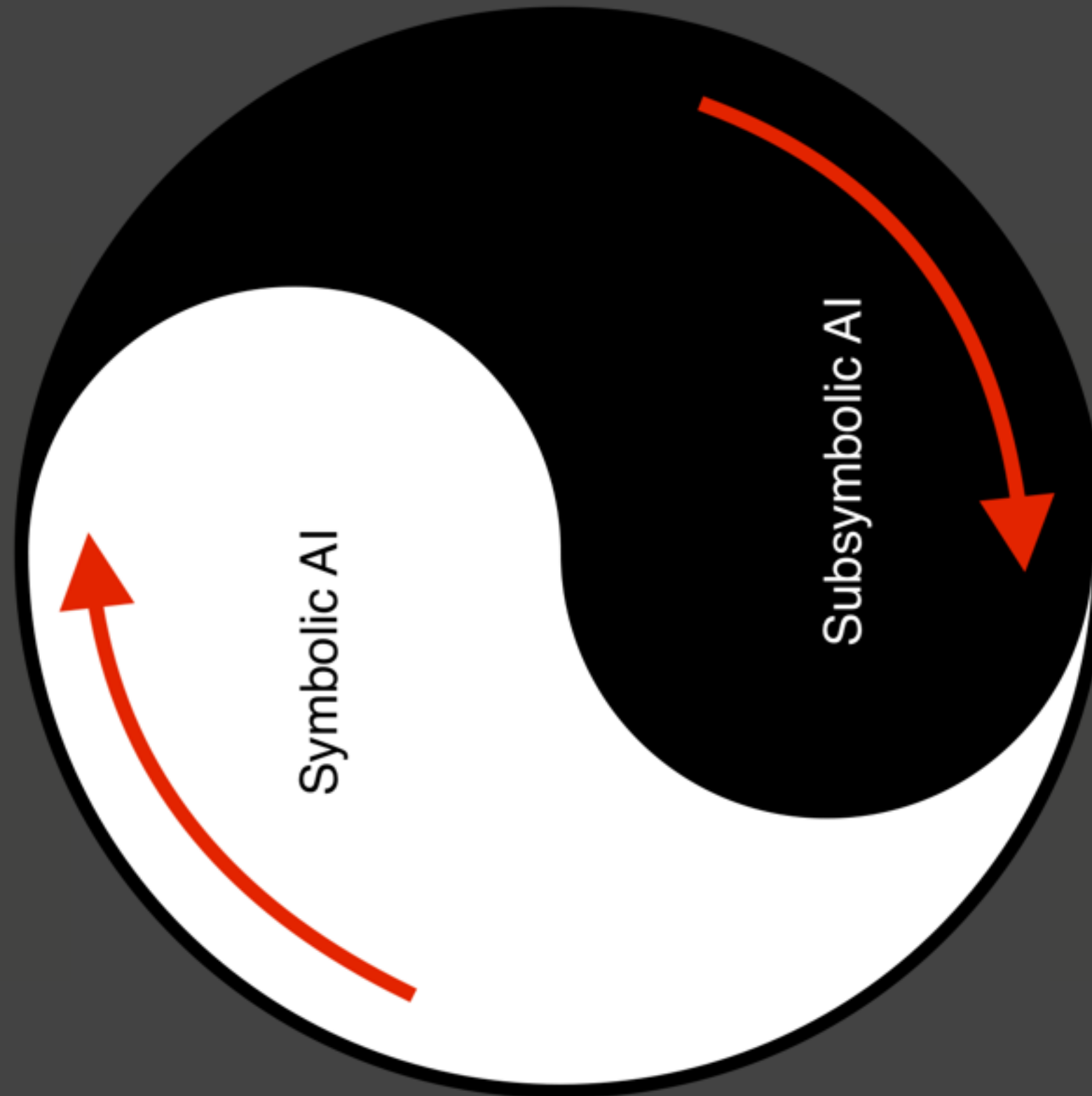
subsymbolic

Brain
Learning
Inductive
Massive Data
Correlations
Robust

Artificial Intelligence

symbolic

Mind
Reasoning
Deductive
Little Data
Causalities
Precise



subsymbolic

Brain
Learning
Inductive
Massive Data
Correlations
Robust

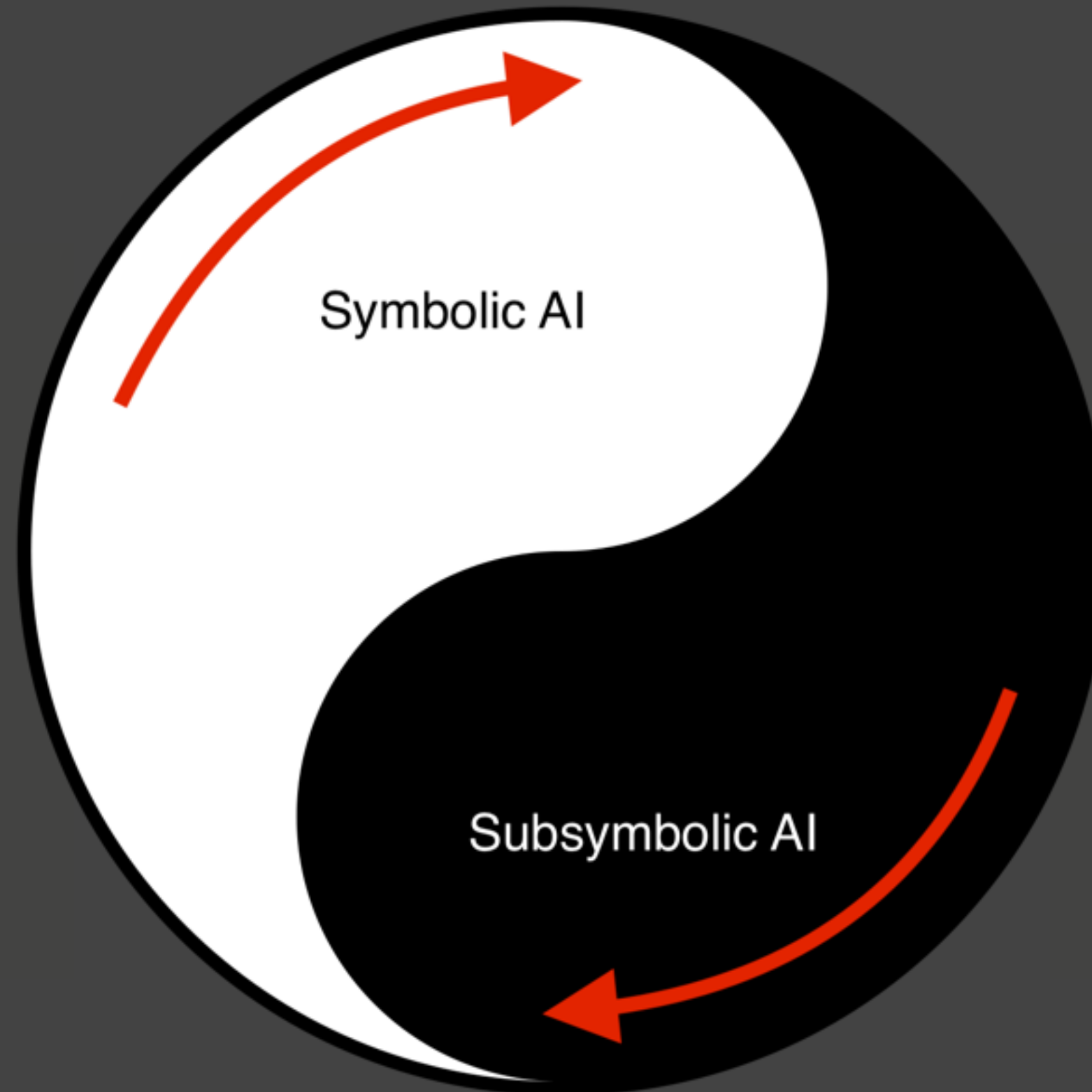
Artificial Intelligence

symbolic

Mind
Reasoning
Deductive
Little Data
Causalities
Precise

subsymbolic

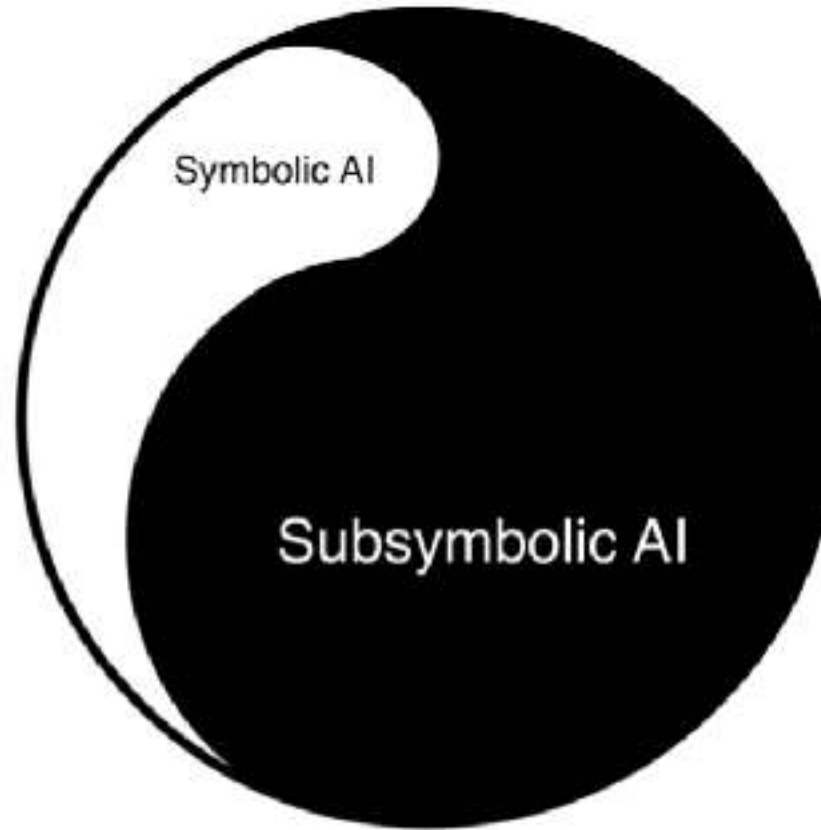
Brain
Learning
Inductive
Massive Data
Correlations
Robust



Examples of success

- ▶ ...
- ▶ SAT-Solver:
solution of open
maths problems
- ▶ ...

(symbolic AI)



Examples of success

- ▶ ...
- ▶ AlphaGo &
AlphaZero: world
champion Chess
and Go
- ▶ ...

(subsymbolic KI)

nature International weekly journal of science

Home | News & Comment | Research | Careers & Jobs | Current Issue | Archive | Audio & Video | For Authors

Archive > Volume 534 > Issue 7605 > News > Article

NATURE | NEWS

Two-hundred-terabyte maths proof is largest ever

A computer cracks the Boolean Pythagorean triples problem — but is it really maths?

Evelyn Lamb

26 May 2016

PDF Rights & Permissions

ALPHA GO

TRIBECA FILM FESTIVAL 2017
BFI LONDON FILM FESTIVAL OFFICIAL SELECTION
WINNER TORONTO INTERNATIONAL FILM FESTIVAL 2017
OFFICIAL SELECTION MILL VALLEY FILM FESTIVAL 2017
OFFICIAL SELECTION NINGBO FILM FESTIVAL
VIFF OFFICIAL SELECTION 2017

The poster shows two men sitting at a table, playing a board game. The background is dark with white dots arranged in a pattern.

Another Interesting Story — Formal Logic in Maths

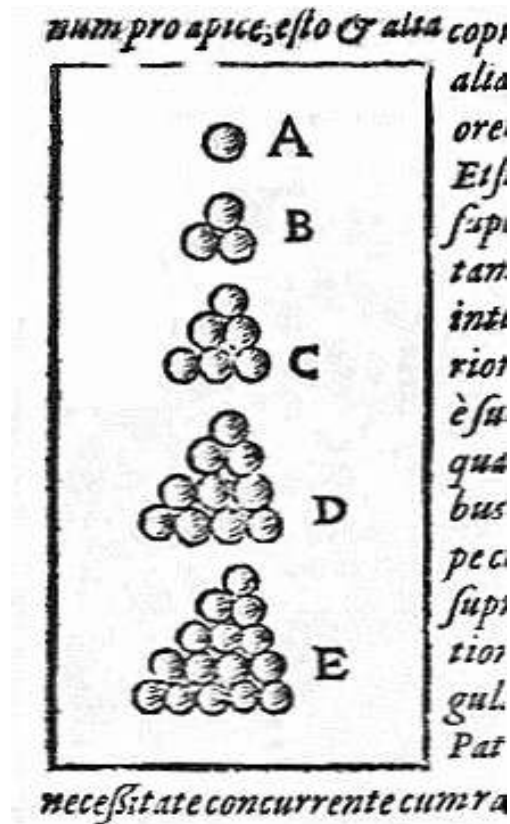


Kepler (1571-1630)



The most compact way of stacking balls of the same size in space is a pyramid.

$$V_{-} = \frac{\pi}{\sqrt{18}} \approx 74\%$$



- ▶ Proved in 1998 by Hales: 300 page proof, with code and data
- ▶ Submitted to the Annals of Mathematics: referees gave up to verify it all
- ▶ **Flyspeck project (completed in 2014):**
 - ▶ A formal verification of the proof in HOL Light
 - ▶ 27,223 proved theorems, 228 definitions, 30 person-years
- ▶ **Work of e.g. Cezary Kalyszik & Josef Urban:**
Automated Theorem Proving assisted by **Machine Learning**
Result: more than 65% of the proofs can actually be fully automated

Outline

- Motivation

- **HOL** — Classical Higher-Order Logic

- Non-Classical Logics (why Classical Logic is not enough)

- **Universal Meta-Logical Reasoning** (why Classical Logic is enough)

- **LogiKEy** — Flexible KR&R Methodology

- Applications:

- Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

Classical Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X p(f(X))$
- Functions	—	✓	$\forall F p(F(a))$
- Predicates/Sets/Rels	—	✓	$\forall P P(f(a))$
Unnamed			
- Functions	—	✓	$\lambda X X$
- Predicates	—	✓	$\lambda X (X = X)$
Statements about			
- Funcs/Preds/Sets/Rels	—	✓	$reflexive \leftrightarrow$
Powerful definitions	—	✓	$reflexive := \lambda R \forall X (R X X)$

Classical Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X p(f(X))$
- Functions	-	✓	$\forall F p(F(a))$
- Predicates/Sets/Rels	-	✓	$\forall P P(f(a))$
Unnamed			
- Functions	-	✓	$\lambda X X$
- Predicates	-	✓	$\lambda X (X = X)$
Statements about			
- Funcs/Preds/Sets/Rels	-	✓	$reflexive \leftrightarrow$
Powerful definitions	-	✓	$reflexive := \lambda R \forall X (R X X)$

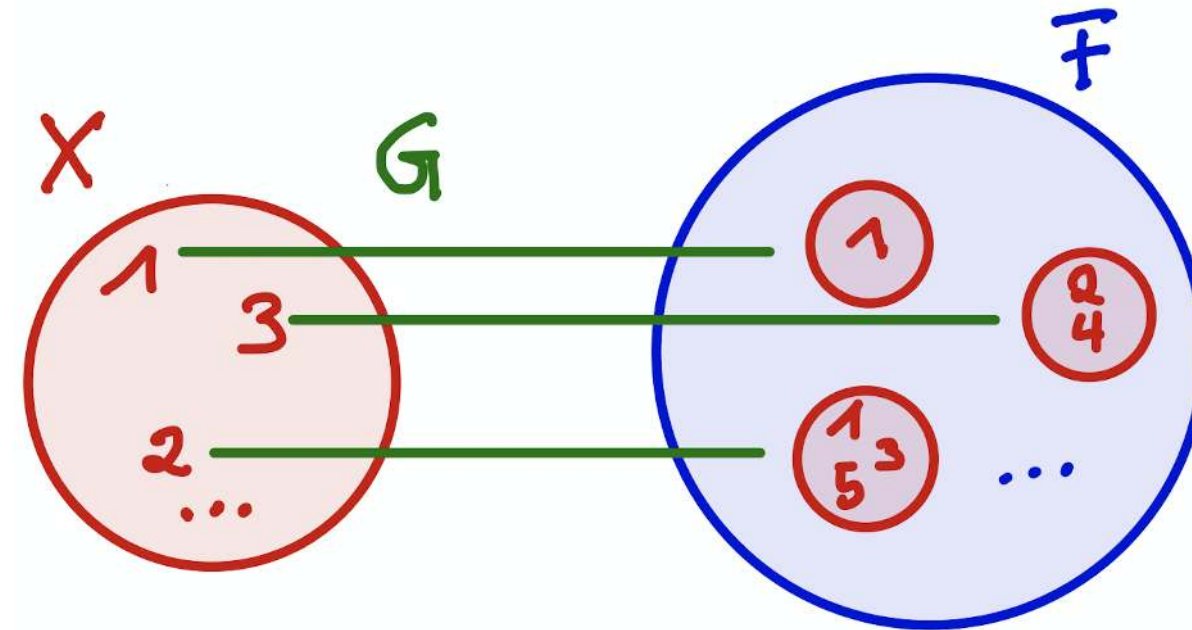
Classical Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X_i p_{i \rightarrow o}(f_{i \rightarrow i}(X_i))$
- Functions	-	✓	$\forall F_{i \rightarrow i} p_{i \rightarrow o}(F_{i \rightarrow o}(a_i))$
- Predicates/Sets/Rels	-	✓	$\forall P_{i \rightarrow o} P_{i \rightarrow o}(f_{i \rightarrow i}(a_i))$
Unnamed			
- Functions	-	✓	$\lambda X_i X_i$
- Predicates	-	✓	$\lambda X_i (X = X)_i$
Statements about			
- Funcs/Preds/Sets/Rels	-	✓	$reflexive_{(o \rightarrow o \rightarrow o) \rightarrow o} \leftrightarrow_{o \rightarrow o \rightarrow o}$
Powerful definitions	-	✓	$reflexive_{(\alpha \rightarrow \alpha \rightarrow o) \rightarrow o} :=$ $\lambda R_{(\alpha \rightarrow \alpha \rightarrow o)} \forall X_\alpha (R X X)$

Types: Prevent paradoxes and inconsistencies

HOL: Expressivity Matters — Surjective Cantor Theorem —

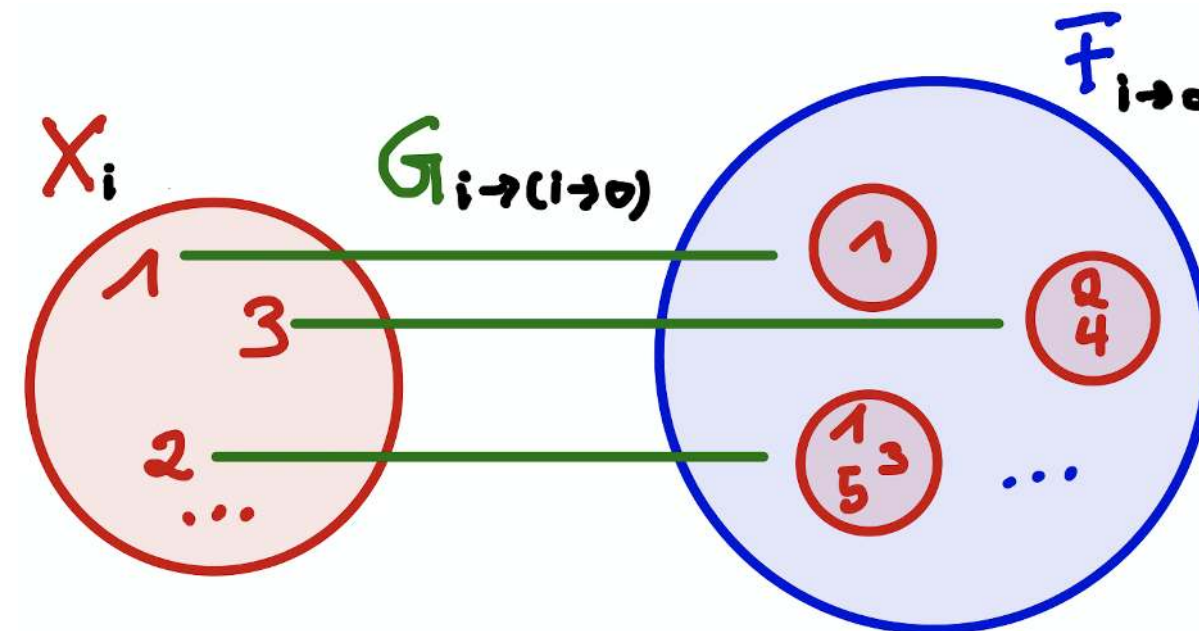
Theorem: *There is no surjective **map** from a **set** to its **powerset**.*



$$\neg \exists G \forall F \exists x \ G(x) = F$$

HOL: Expressivity Matters — Surjective Cantor Theorem —

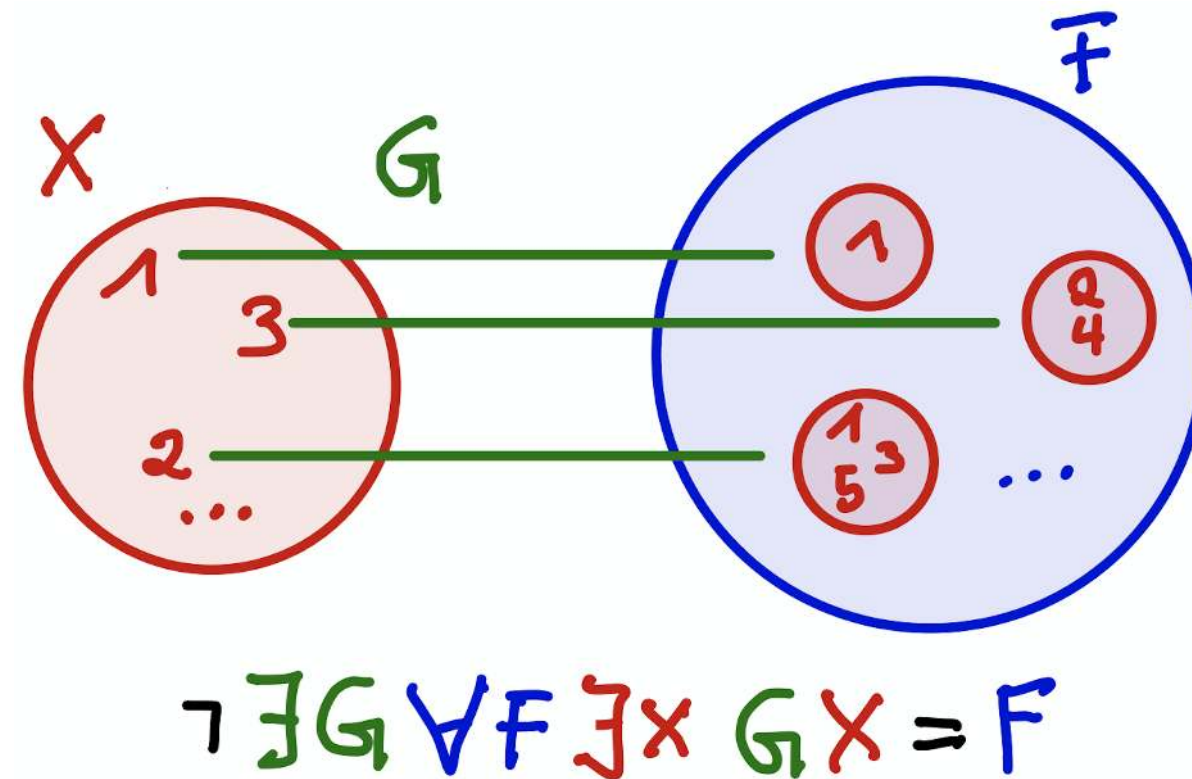
Theorem: *There is no surjective **map** from a **set** to its **powerset**.*



$$\neg \exists G_{i \rightarrow (i \rightarrow 0)} \forall P_{i \rightarrow 0} \exists X_i G X = P$$

HOL: Expressivity Matters — Surjective Cantor Theorem —

Theorem: *There is no surjective **map** from a **set** to its **powerset**.*



Proof (by contradiction):

Assume there is a surjective G .

Consider the following choice for F : $F := \{X \mid \neg X \in (G X)\}$

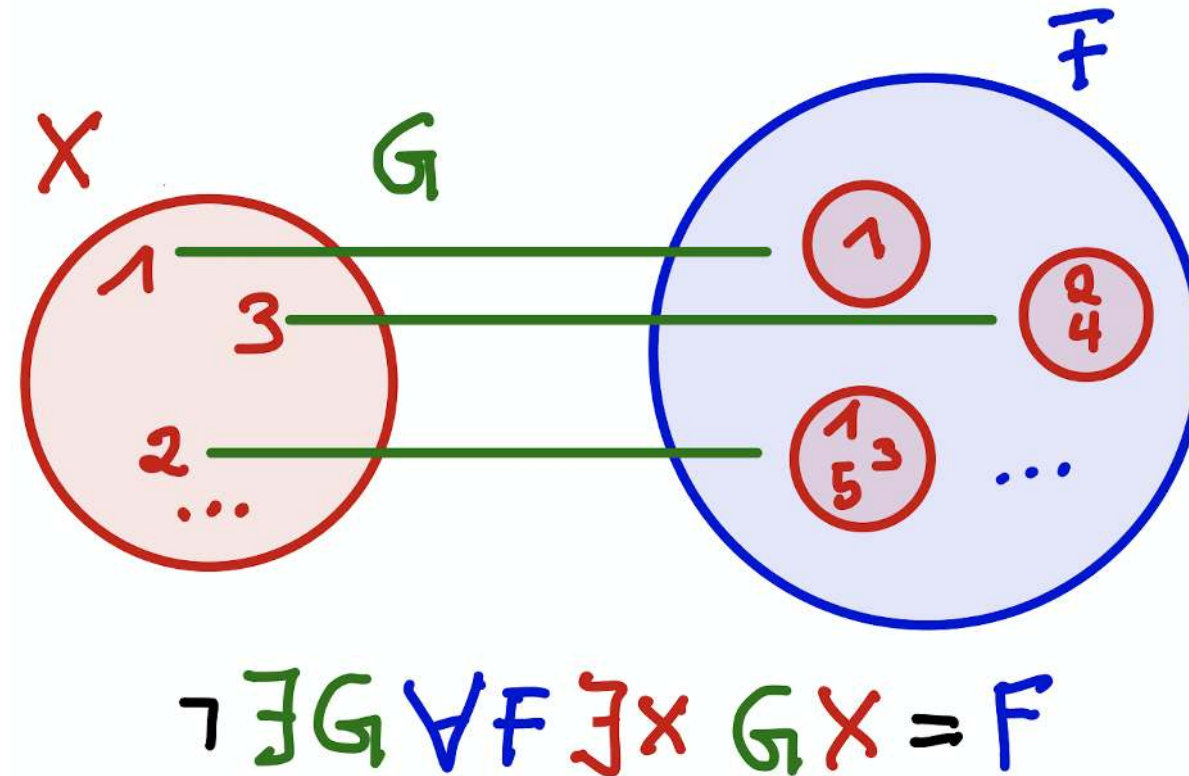
Then there is an a such that $(G a) = F$ (since G is surj. by ass.)

But then: $a \in (G a)$ iff $a \in F$ iff $\neg a \in (G a)$ (by def. of F)

Contradiction. $\square$

HOL: Expressivity Matters — Surjective Cantor Theorem —

Theorem: *There is no surjective **map** from a **set** to its **powerset**.*



HOL ATPs can solve this problem very efficiently

In our **Leo provers** this works by a combination of (heuristic) **guessing** and straightforward **resolution and HO unification**:

“ $\neg$ ” in $F := \{X \mid \neg X \in (G X)\}$ is **guessed**, rest is straightforward

Further reading: [AndrewsEtAl., Automating higher-order logic, 1984]

HOL: Expressivity Matters — Boolos' Curious Inference —

1. $\forall n f(n, 1) = s(1)$
2. $\forall x f(1, s(x)) = s(s(f(1, x)))$
3. $\forall n \forall x f(s(n), s(x)) = f(n, f(s(n), x))$
4. $D(1)$
5. $\forall x D(x) \rightarrow D(s(x))$
- $\vdots$
6. $D(f(s(s(s(s(1))))), s(s(s(s(1)))))$

[George Boolos, A curious inference, J.Phil.Log., 16:1-12, 1987]

Proof in FOL? — yes, **but practically infeasible** number of proof steps

Proof in HOL? — yes, on **one single page**

► Key: powerful lemmata as instances of comprehension axioms

See our interactive verification of Boolos's proof:

[BenzmüllerBrown, The curious inference of Boolos in MIZAR and OMEGA, 2007]

HOL: Expressivity Matters — Boolos' Curious Inference —

1. $\forall n f(n, 1) = s(1)$
2. $\forall x f(1, s(x)) = s(s(f(1, x)))$
3. $\forall n \forall x f(s(n), s(x)) = f(n, f(s(n), x))$
4. $D(1)$
5. $\forall x D(x) \rightarrow D(s(x))$
- $\vdots$
6. $D(f(s(s(s(s(1))))), s(s(s(s(1)))))$

[George Boolos, A curious inference, J.Phil.Log., 16:1-12, 1987]

Proof in FOL? — yes, **but practically infeasible** number of proof steps

Proof in HOL? — yes, on **one single page**

- ▶ Key: powerful lemmata as instances of comprehension axioms

See our interactive verification of Boolos's proof:

[BenzmüllerBrown, The curious inference of Boolos in MIZAR and OMEGA, 2007]

Proof has recently been put into Isabelle/HOL: [Kettland, AFP, 2022]

Boolos Curious Inference

STUDIES IN LOGIC, GRAMMAR AND RHETORIC 10 (23) 2007

The Curious Inference of Boolos in Mizar and OMEGA*

Christoph E. Benz Müller^{1,2} and Chad E. Brown²

¹The University of Cambridge, Cambridge, UK

²Universität des Saarlandes, Saarbrücken, Germany
chris@brown@ags.uni-sb.de

To Andrzej Trybulec

Abstract. We examine Boolos' curious inference and formalize it in a system based on set theory (Mizar) and a system based on classical higher-order logic (OMEGA). The Boolos example is interesting because while it can in principle be proven using a complete first-order calculus, it is impractical to do so. In our case study we are interested in aspects such as how natural and at what level of granularity Boolos' short second-order proof sketch can be formalized in Mizar and OMEGA.

Proof has recently been
put into Isabelle/HOL:
[Kettland, AFP, 2022]

JOURNAL ARTICLE A Curious Inference

George Boolos



Journal of Philosophical
Logic
Vol. 16, No. 1 (Feb., 1987),
pp. 1-12 (12 pages)
Published by: Springer

We first present a sketch of a second-order derivation of (6) from (1) – (5) of which a complete formalization in any standard axiomatic formulation of second-order logic can easily be written out:

By the comprehension principle of second-order logic,
 $\exists N \forall z (Nz \leftrightarrow \forall X [X1 \ \& \ \forall y (Xy \rightarrow Xsy) \rightarrow Xz])$, and then for some N ,
 $\exists E \forall z (Ez \leftrightarrow Nz \ \& \ Dz)$.

LEMMA 1: $N1; \forall y (Ny \rightarrow Nsy); Nssss1; E1; \forall y (Ey \rightarrow E sy); Es1$.

LEMMA 2: $\forall n (Nn \rightarrow \forall x (Nx \rightarrow Efnx))$.

Proof. By comprehension, $\exists M \forall n (Mn \leftrightarrow \forall x (Nx \rightarrow Efnx))$. We want $\forall n (Nn \rightarrow Mn)$. Enough to show $M1$ and $\forall n (Mn \rightarrow Msn)$, for then if Nn, Mn .

$M1$: Want $\forall x (Nx \rightarrow Ef1x)$. By comprehension, $\exists Q \forall x (Qx \leftrightarrow Ef1x)$. Want $\forall x (Nx \rightarrow Qx)$. Enough to show $Q1$ and $\forall x (Qx \rightarrow Qsx)$.

$Q1$: Want $Ef11$. But $f11 = s1$ by (1) and $Es1$ by Lemma 1.
 $\forall x (Qx \rightarrow Qsx)$: Suppose Qx , i.e. $Ef1x$. By (2) $f1sx = ssf1x$; by Lemma 1 twice, $Ef1sx$. Thus Qsx and $M1$.

$\forall n (Mn \rightarrow Msn)$: Suppose Mn , i.e. $\forall x (Nx \rightarrow Efnx)$. Want Msn , i.e. $\forall x (Nx \rightarrow Efsnx)$. By comprehension, $\exists P \forall x (Px \leftrightarrow Efsnx)$. Want $\forall x (Nx \rightarrow Px)$. Enough to show $P1$ and $\forall x (Px \rightarrow Psx)$.

$P1$: Want $Efsn1$. But $fsn1 = s1$ by (1) and $Es1$ by Lemma 1.

$\forall x (Px \rightarrow Psx)$: Suppose Px , i.e. $Efsnx$; thus $Nfsnx$. Want $Efsnsx$. Since $Nfsnx$ and Mn , $Efnfsnx$. But by (3) $fnfsnx = fsnsx$; thus $Efsnsx$. By Lemma 1, $Nssss1$. By Lemma 2, $Efssss1ssss1$. Thus $Dfssss1ssss1$, as desired.

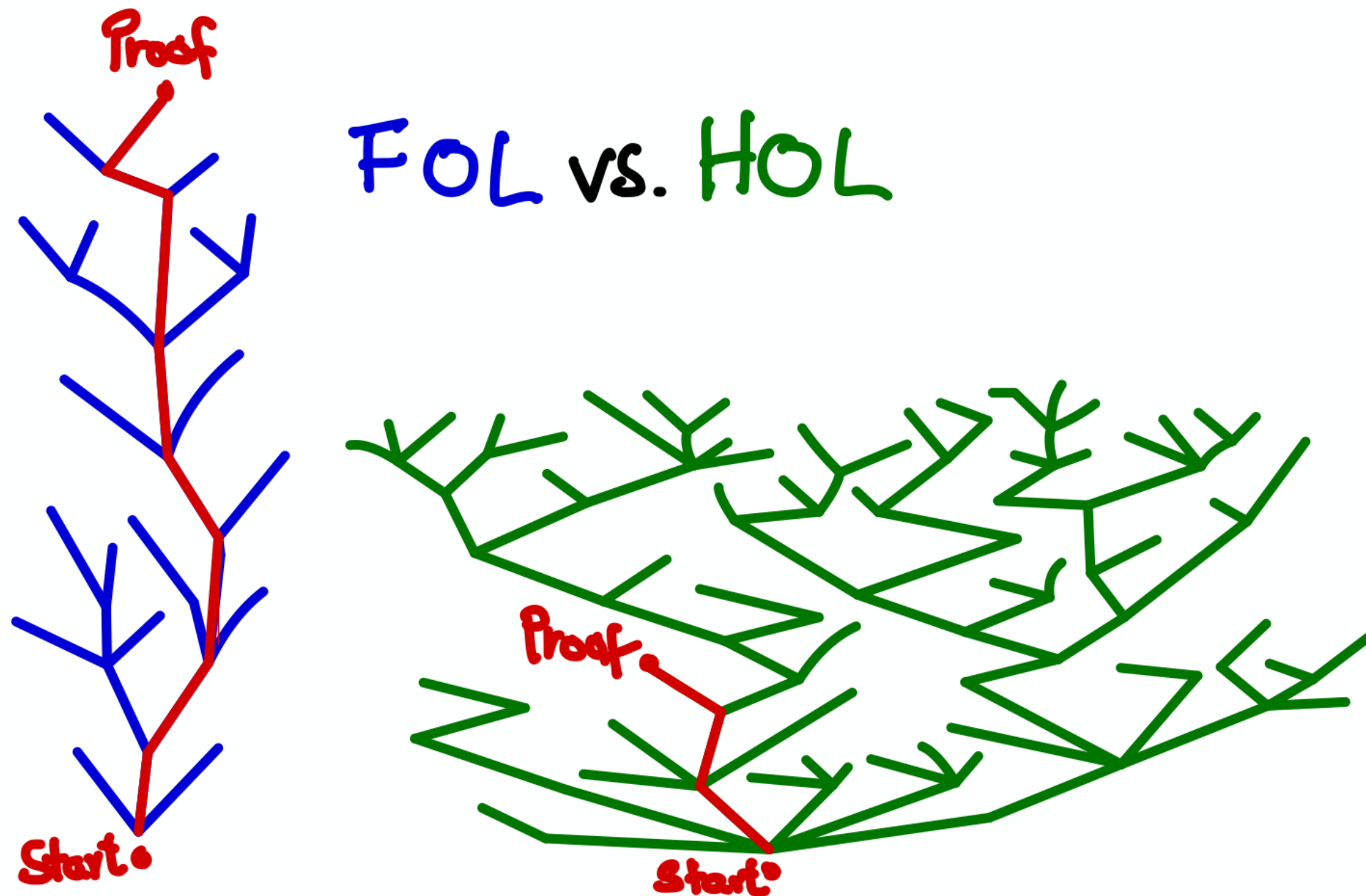
Screenshot

New(!): Automating Boolos Curious Inference

```
1 theory BoolosATP imports Main
2 begin
3
4 typedecl i
5 consts
6   e :: "i"    (* one *)
7   s :: "i⇒i"   (* successor function *)
8   F :: "i⇒i⇒i" (* binary function; axiomatised below as Ackermann function *)
9   D :: "i⇒bool" (* arbitrary uninterpreted unary predicate *)
10
11 axiomatization where
12   A1: "∀n. F n e = s e"    (* Axiom for Ackermann function F *)
13 and A2: "∀y. F e (s y) = s (s (F e y))" (* Axiom for Ackermann function F *)
14 and A3: "∀x y. F (s x) (s y) = F x (F (s x) y)" (* Axiom for Ackermann function F *)
15 and A4: "D e"    (* D holds for one *)
16 and A5: "∀x. D x → D (s x)" (* if D holds for x it also holds for the successor of x *)
17
18 lemma "D (F (s (s (s (s e)))) (s (s (s (s e)))))" sledgehammer oops (* no proof; hopeless! *)
19
20 definition isIndSet where "isIndSet X ≡ (X e) ∧ (∀x. X x → X (s x))" (* X is inductive *)
21 definition N where "N x ≡ (∀X::i⇒bool. isIndSet X → X x)" (* N is smallest inductive set *)
22 definition P where "P x y ≡ N (F x y)" (* P(x,y) iff F(x,y) is in N *)
23
24 lemma "D (F (s (s (s (s e)))) (s (s (s (s e)))))" (* ATPs can now proof this: using the Defs *)
25   sledgehammer (* proof found *)
26   by (metis A1 A2 A3 A4 A5 N_def P_def isIndSet_def) (* proof reconstruction succeeds *)
27 end
```

Finding: only the invention of definitions is key — rest can be automated!

HOL: Expressivity Matters — Boolos' Curious Inference —



Proofs in HOL can be short, elegant, intuitive

HOL: Sets and Relations

Elegant representations (omitting types) for operations on sets:

$\{x \mid p(x) \text{ and } q(x)\}$	represented as	$\lambda X ((p\ X) \wedge (q\ X))$
$\in$	represented as	$\lambda X \lambda M (M\ X)$
$\cap$	represented as	$\lambda M \lambda N (\lambda X (M\ X \wedge N\ X))$
$\cup$	represented as	$\lambda M \lambda N (\lambda X (M\ X \vee N\ X))$
$\subseteq$	represented as	$\lambda M \lambda N (\forall X (M\ X \rightarrow N\ X))$
...		

Advantage: Axiomatisation of sets can (usually) be avoided in HOL

Very useful, e.g., for semantic embeddings of other logics in HOL

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } \equiv_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } =_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$s, t ::= \boxed{p_\alpha} \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad$ (or only $=_{\alpha \rightarrow \alpha \rightarrow o}$)

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid \boxed{X_\alpha} \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } =_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$
constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad$ (or only $=_{\alpha \rightarrow \alpha \rightarrow o}$)

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid \boxed{(s_{\alpha \rightarrow \beta} t_\alpha)_\beta}$
constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad$ (or only $=_{\alpha \rightarrow \alpha \rightarrow o}$)

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } =_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } =_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Simple Syntax

Simple Types: $\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$
(we may add further base types; types are often not displayed)

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o} \quad (\text{or only } =_{\alpha \rightarrow \alpha \rightarrow o})$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ $\perp, \top, \rightarrow, \leftrightarrow, \exists, =$ can be defined: e.g., $\exists X_\alpha s_o$ stands for $\neg \forall X_\alpha \neg s_o$

HOL is a language of terms. Terms of type o are called formulas.

HOL: Example Formula — Surjective Cantor Theorem —

Theorem: $\neg \exists G_{i \rightarrow (i \rightarrow o)} \forall F_{i \rightarrow o} \exists X_i G X = F$

represented in HOL (as defined on previous slide) as

$$\neg \neg \Pi(\lambda G_{i \rightarrow (i \rightarrow o)} \neg (\Pi(\lambda F_{i \rightarrow o} \neg \Pi(\lambda X_i \neg G X = F))))$$

This is e.g. also the *internal representation* in our Leo provers.

HOL: Example Formula — Surjective Cantor Theorem —

Theorem: $\neg \exists G_{i \rightarrow (i \rightarrow o)} \forall F_{i \rightarrow o} \exists X_i G X = F$

represented in HOL (as defined on previous slide) as

$$\neg \neg \Pi(\lambda G_{i \rightarrow (i \rightarrow o)} \neg (\Pi(\lambda F_{i \rightarrow o} \neg \Pi(\lambda X_i \neg G X = F))))$$

This is e.g. also the *internal representation* in our Leo provers.

However, *intuitive user representations* are supported in HOL provers (e.g. in the Isabelle/HOL system):

lemma SurjectiveCantor: " $\neg (\exists G :: i \Rightarrow i \Rightarrow \text{bool}. \forall F. \exists X. G X = F)$ "

HOL: Example Formula — Surjective Cantor Theorem —

Theorem: $\neg \exists G_{i \rightarrow (i \rightarrow o)} \forall F_{i \rightarrow o} \exists X_i G X = F$

represented in HOL (as defined on previous slide) as

$$\neg \neg \Pi(\lambda G_{i \rightarrow (i \rightarrow o)} \neg (\Pi(\lambda F_{i \rightarrow o} \neg \Pi(\lambda X_i \neg G X = F))))$$

This is e.g. also the *internal representation* in our Leo provers.

However, *intuitive user representations* are supported in HOL provers (e.g. in the Isabelle/HOL system):

lemma SurjectiveCantor: " $\neg (\exists G :: i \Rightarrow i \Rightarrow \text{bool}. \forall F. \exists X. G X = F)$ "

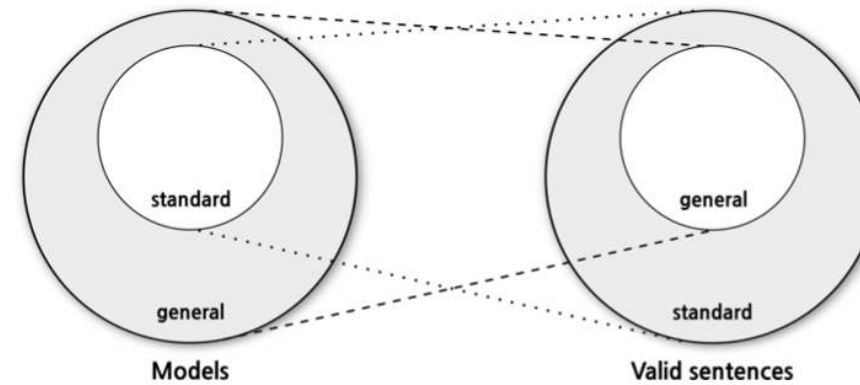
```
proof
  assume 1: " $\exists G :: i \Rightarrow i \Rightarrow \text{bool}. \forall F. \exists X. G X = F$ "
  obtain g :: "i  $\Rightarrow$  i  $\Rightarrow$  bool" where 2: " $\forall F. \exists X. g X = F$ " using 1 by blast
  let ?F = " $\lambda X. \neg g X X$ " (* choose F = {x |  $\neg (x \in (g x))$ } *)
  have 3: " $\exists Y. ?F = g Y$ " using 2 by metis
  obtain a :: i where 4: "?F = g a" using 3 by blast
  have 5: " $g a a \longleftrightarrow ?F a$ " using 4 by metis
  have 6: " $g a a \longleftrightarrow \neg g a a$ " using 5 by metis
  show False using 6 by blast
qed
```

HOL: Well Understood Semantics

HOL with standard semantics:

HOL with Henkin's general semantics:

incomplete
semi-decidable & compact



more model structures ... fewer valid formulas

Important principles are still valid in Henkin's general models:

► Comprehension (type-restricted):

$$\forall G \exists F \forall \overline{X^n} F \overline{X^n} = G$$

► Boolean Extensionality:

$$\forall P \forall Q ((P \leftrightarrow Q) \rightarrow P = Q)$$

► Functional Extensionality:

$$\forall F \forall G ((\forall X F X = G X) \rightarrow F = G)$$

Note: Any “Henkin-valid” formula is also valid in standard semantics!

Suggested Reading

► Origin

[Church, JSL, 1940]

► Henkin's general semantics:

[Henkin, JSL, 1950] [Andrews, JSL, 1971, 1972]

► Extensionality&Intensionality:

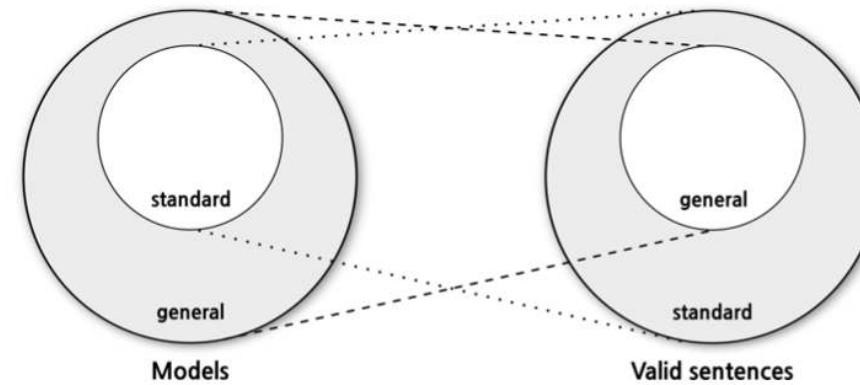
[BenzmüllerEtAl., JSL, 2004] [Muskens, JSL, 2007]

HOL: Well Understood Semantics

HOL with standard semantics:

HOL with Henkin's general semantics:

incomplete
semi-decidable & compact



more model structures ... fewer valid formulas

Important principles are still valid in Henkin's general models:

► Comprehension (type-restricted):

$$\forall G \exists F \forall \overline{X^n} F \overline{X^n} = G$$

► Boolean Extensionality:

$$\forall P \forall Q ((P \leftrightarrow Q) \rightarrow P = Q)$$

► Functional Extensionality:

$$\forall F \forall G ((\forall X F X = G X) \rightarrow F = G)$$

Note: Any “Henkin-valid” formula is also valid in standard semantics!

Suggested Reading

► Origin

[Church, JSL, 1940]

► Henkin's general semantics:

[Henkin, JSL, 1950] [Andrews, JSL, 1971, 1972]

► Extensionality&Intensionality:

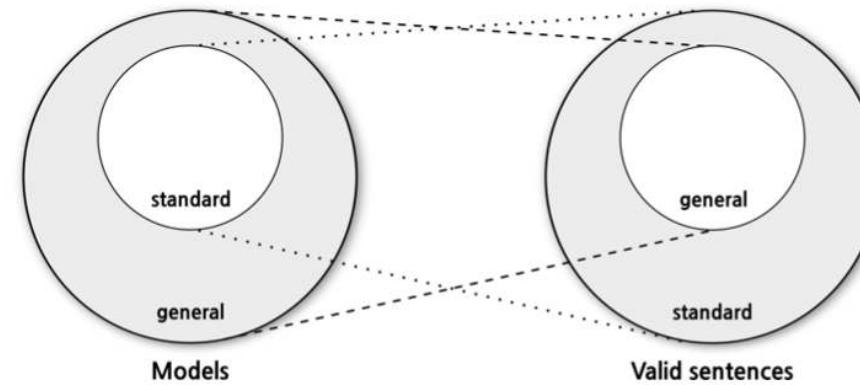
[BenzmüllerEtAl., JSL, 2004] [Muskens, JSL, 2007]

HOL: Well Understood Semantics

HOL with standard semantics:

HOL with Henkin's general semantics:

incomplete
semi-decidable & compact



more model structures ... fewer valid formulas

Important principles are still valid in Henkin's general models:

► Comprehension (type-restricted):

$$\forall G \exists F \forall \overline{X^n} F \overline{X^n} = G$$

► Boolean Extensionality:

$$\forall P \forall Q ((P \leftrightarrow Q) \rightarrow P = Q)$$

► Functional Extensionality:

$$\forall F \forall G ((\forall X F X = G X) \rightarrow F = G)$$

Note: Any “Henkin-valid” formula is also valid in standard semantics!

Suggested Reading

► Origin

[Church, JSL, 1940]

► Henkin's general semantics:

[Henkin, JSL, 1950] [Andrews, JSL, 1971, 1972]

► Extensionality&Intensionality:

[BenzmüllerEtAl., JSL, 2004] [Muskens, JSL, 2007]

[PhD thesis by Steen]

HOL: Theorem Provers

Since 2008: TPTP Infrastructure for HOL ATP (with Sutcliffe and others)

Increasing activities in HOL ATP since then:

TPS	CMU, US	1980s–2000s	Mating Method
LEO-I	U Saarbrücken, DE	1995–2006	Resolution
LEO-II	Cambridge U, UK	2007–	Paramodulation
Satallax	Saarbr./Paris-Saclay, FR	2010–	Tableaux
IsabelleP	Isabelle Team, DE/UK/NL	2010–	Misc
Leo-III	Berlin/Luxemb., DE/LU	2013–	Ordered Paramodulation
CoqATP	Coq Team, FR	2013–	Saturation Algorithm
agsyHOL	Chalmers, SE	2014–	Lazy Narrowing
HOLyHammer	Innsbr./Praha, AU/CZ	2014–	Misc
Zipperpin	Vrije U Amsterdam, NL	2017–	Superposition
CVC4	U Iowa, US	2019–	SMT Solver
Vampire	U Manchester, UK	2020–	Superposition
...	...	...	...

Model finders: Nitpick, Nunchaku

Proof assistants: Isabelle, HOL4, HOL light, PVS, Coq, IMPS, Lean, NuprL, ...

Survey articles:
[BenzmüllerMiller, Automation of Higher-Order Logic, HB of History Of Logic, 2014]
[BenzmüllerAndrews, Church’s Type Theory, SEP, 2019]



HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg (p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg (p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg (p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg (p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg(p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg(p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg(p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg(p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg(p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg(p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg(p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg(p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Signature contains: $f_{i \rightarrow i}$ $p_{i \rightarrow o}$

Conjecture: $\forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Negated Conjecture: $\neg \forall X_i \exists H_{i \rightarrow i} ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\exists X_i \forall H_{i \rightarrow i} \neg ((p (H (f X))) \rightarrow (p (f (H X))))$

Clause normalisation: $\neg ((p (H (f s))) \rightarrow (p (f (H s))))$

Clause normalisation: ...

Clause normalisation: $c1 : (p (H (f s)))$ $c2 : \neg(p (f (H s)))$

Resolution: $c3 : \perp$ if $(p (H (f s))) \stackrel{?}{=} (p (f (H s)))$

Unification: $H \leftarrow \lambda Y_i Y$

Unification: $H \leftarrow \lambda Y_i (f Y)$

Unification: $H \leftarrow \lambda Y_i (f (f Y))$

Unification: $H \leftarrow \dots$

Unification in HOL: **undecidable & infinitary** (but often harmless in practice)

Unification in HOL mod. Boolean extensionality: $c1 : (p (a \wedge b))$ $c2 : \neg(p (b \wedge a))$

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: Primitive Substitution is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: Primitive Substitution is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: Primitive Substitution is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R\ X\ Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R\ X\ Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R\ X\ Y)$

Clause normalisation: $\neg \neg(R\ s\ t)$

Clause normalisation: $(R\ s\ t)$

Only clause: $c1 : (R\ s\ t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp)\ s\ t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: Primitive Substitution is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: **Primitive Substitution** is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: **Primitive Substitution** is key to the automation of Cantor's theorem

HOL: Some Simple Examples (Resolution)

Conjecture: $\forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Negated Conjecture: $\neg \forall X_i \forall Y_i \exists R_{i \rightarrow i \rightarrow o} \neg(R X Y)$

Clause normalisation: $\exists X_i \exists Y_i \forall R_{i \rightarrow i \rightarrow o} \neg \neg(R X Y)$

Clause normalisation: $\neg \neg(R s t)$

Clause normalisation: $(R s t)$

Only clause: $c1 : (R s t)$

Primitive Substitution: $c2 : ((\lambda X_i \lambda Y_i \perp) s t)$ (i.e. $R \longleftarrow$ empty relation)

β -normalisation: $c3 : \perp$

Obvious: clause normalisation generally needed over and over again

Remark: **Primitive Substitution** is key to the automation of Cantor's theorem

Primitive Substitution as Eureka Step: Example

Studies in Computational Metaphysics:

Gödel's (1970) Ontological Argument for Existence of God

Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-16)

The Inconsistency in Gödel's Ontological Argument: A Success Story for AI in Metaphysics

Christoph Benz Müller*

Freie Universität Berlin & Stanford University
c.benzmueller@gmail.com

Bruno Woltzenlogel Paleo

Australian National University
bruno.wp@gmail.com

Primitive Substitution: is **key step** in LEO-II's proof of inconsistency!

(However, we also verified Dana Scott's variant and showed consistency)

Automating Boolos Curious Inference?

```
1 theory BoolosATP imports Main
2 begin
3
4 typedecl i
5 consts
6 e :: "i" (* one *)
7 s :: "i⇒i" (* successor *)
8 F :: "i⇒i⇒i" (* binary function; axiomatised below
9 D :: "i⇒bool" (* arbitrary uninterpreted unary predi
10
11 axiomatization where
12   A1: "∀n. F n e = s e" (* Axiom for Ackermann funct
13 and A2: "∀y. F e (s y) = s (s (F e y))" (* Axiom for A
14 and A3: "∀x y. F (s x) (s y) = F x (F (s x) y)" (* Axi
15 and A4: "D e" (* D holds for one *)
16 and A5: "∀x. D x → D (s x)" (* if D holds for x it a
17
18
19 definition isIndSet where "isIndSet X ≡ (X e) ∧ (∀x. X
20 definition N where "N x ≡ (∀X::i⇒bool. isIndSet X →
21 definition P where "P x y ≡ N (F x y)" (* P(x,y) iff
22
23 lemma "D (F (s (s (s (s e)))) (s (s (s (s e))))))" (* A
24   sledgehammer (* proof found *)
25   by (metis A1 A2 A3 A4 A5 N_def P_def isIndSet_def)
26 end
```

A Lost Proof

A Lost Proof, Benzmüller, C. & Kerber, M. in
Proceedings of the IJCAR 2001 Workshop:
Future Directions in Automated Reasoning
pages 13-24, Siena, Italy, 2001.

Christoph Benzmüller¹ and Manfred Kerber²

¹ Fachrichtung Informatik, Universität des Saarlandes

66041 Saarbrücken, Germany

chris@ags.uni-sb.de

² School of Computer Science

The University of Birmingham, Birmingham B15 2TT, England

M.Kerber@cs.bham.ac.uk

Abstract. We re-investigate a proof example presented by George Boolos which perspicuously illustrates Gödel's argument for the potentially drastic increase of proof-lengths in formal systems when carrying through the argument at too low a level. More concretely, restricting the order of the logic in which the proof is carried through to the order of the logic in which the problem is formulated in the first place can result in unmanageable long proofs, although there are short proofs in a logic of higher order.

Our motivation in this paper is of practical nature and its aim is to sketch the implications of this example to current technology in automated theorem proving, to point to related questions about the foundational character of type theory (without explicit comprehension axioms) for mathematics, and to work out some challenging aspects with regard to the automation of this proof – which, as we believe, nicely illustrates the discrepancy between the creativity and intuition required in mathematics and the limitations of state of the art theorem provers.

Finding: the invention of definitions is

Automating Boolos Curious Inference?!

```
% Declarations
thf(e_decl, type, e : $i).
thf(s_decl, type, s : $i > $i).
thf(d_decl, type, d : $i > $o).
thf(f_decl, type, f : $i > $i > $i).
thf(isIndSet_decl, type, isIndSet : ($i > $o) > $o).
thf(n_decl, type, n : $i > $o).
thf(p_decl, type, p : $i > $i > $o).

% Axioms
thf(a1, axiom, ((![N : $i]: ((f @ N @ e) = (s @ e))))).
thf(a2, axiom, ((![Y : $i]: ((f @ e @ (s @ Y)) = (s @ (s @ (f @ e @ Y)))))).
thf(a3, axiom, ((![X : $i, Y : $i]: ((f @ (s @ X) @ (s @ Y)) = (f @ X @ (f @ (s @ X) @ Y))))).
thf(a4, axiom, ((d @ e))).
thf(a5, axiom, ((![X : $i]: ((d @ X) => (d @ (s @ X)))))).

% Definitions
thf(isIndSet_def, axiom, ((isIndSet = (^[X3 : $i > $o]: (((X3 @ e)) & ((![X2 : $i]: (((X3 @ X2)) => ((X3 @ (s @ X2)))))))))).
thf(n_def, axiom, ((n = (^[X2 : $i]: (![X3 : $i > $o]: (((isIndSet @ X3)) => ((X3 @ X2))))))).
thf(p_def, axiom, ((p = (^[X2 : $i]: (^[Y2 : $i]: (n @ (f @ X2 @ Y2)))))).

% Conjecture
thf(conj_0, conjecture, ((d @ (f @ (s @ (s @ (s @ (s @ e)))) @ (s @ (s @ (s @ (s @ e)))))))).
```

```
% END OF SYSTEM OUTPUT
% RESULT: SOT_7nZdVA - E---3.0 says Theorem - CPU = 5.85 WC = 0.80
% OUTPUT: SOT_7nZdVA - E---3.0 says CNFRefutation - CPU = 5.85 WC = 0.80
```



Insight: PrimSubst in HOL not sufficient to replace these defining axioms!

(a single cut-strong axiom, see [http://doi.org/10.2168/LMCS-5\(1:6\)2009](http://doi.org/10.2168/LMCS-5(1:6)2009), would suffice though)

Finding: the invention of definitions is key — rest can be automated!

Automating Boolos Curious Inference?!

IDV display of
proof by E 3.0

Settings/Tools ^

Node Information ^

Name: isIndSet_def
Type: thf
Role: axiom
Interestingness: 0.00
Formula: $\text{isIndSet} = (\wedge [X4:\$i>\$o]: ((X4@e) \& ! [X5:\$i]: ((X4@X5) \Rightarrow (X4@(s@X5))))$
Inference Record:

Full TPTP Statement: v

Show/Hide Nodes ^

Compute interestingness for FOF/CNF derivations

Enable interestingness scaling

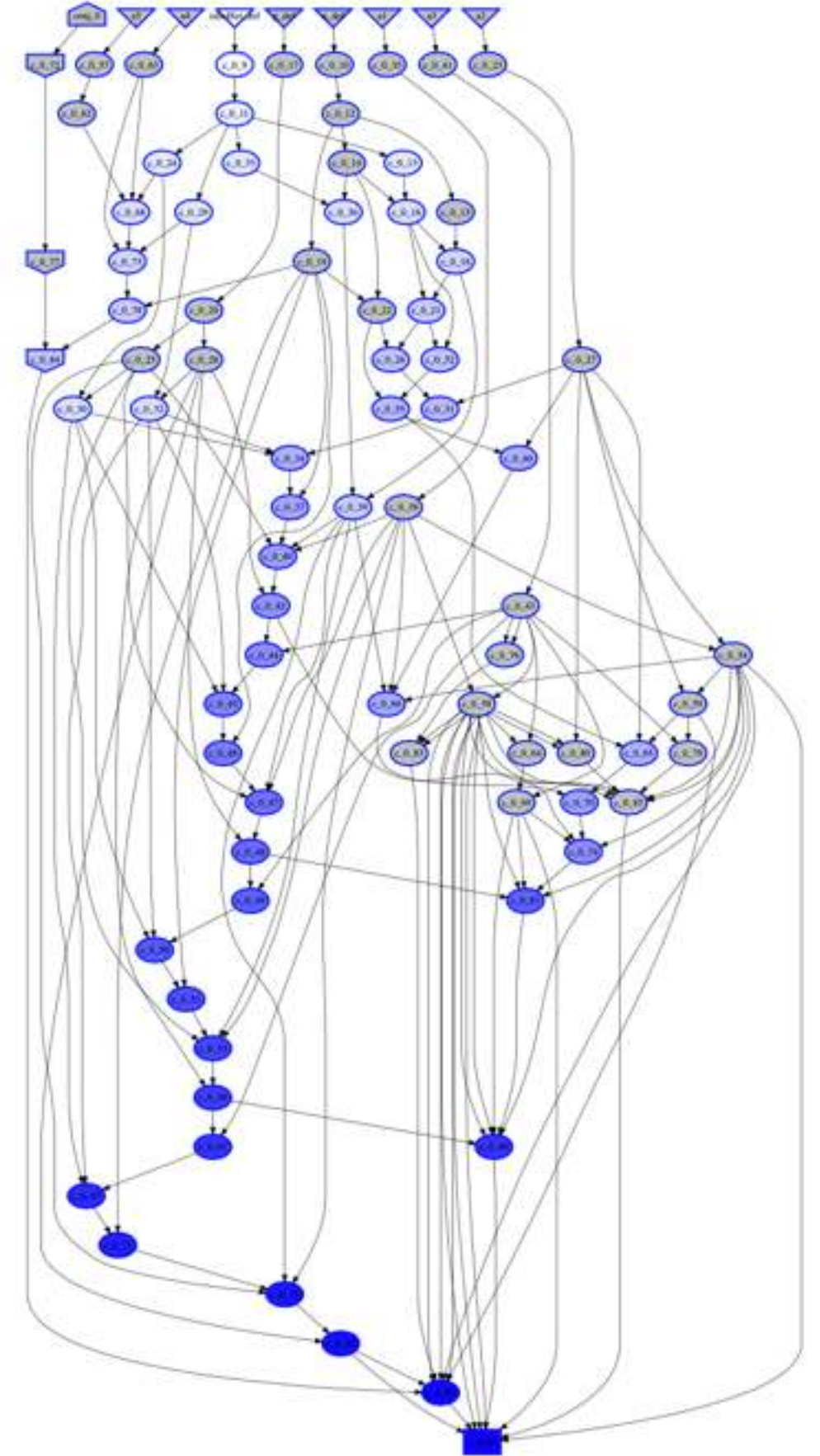
Blacken boring nodes:



Blacken non-lemmas

Remove black nodes

Reset



Outline

- Motivation
- **HOL** — Classical Higher-Order Logic
 - **Non-Classical Logics** (why Classical Logic is not enough)
 - **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy** — Flexible KR&R Methodology
- Applications:
 - Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

$\leftrightarrow$

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter knows “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐Belief ☐Time ☐Obligation ☐...

Many other Non-Classical Logics (besides Modal Logics)

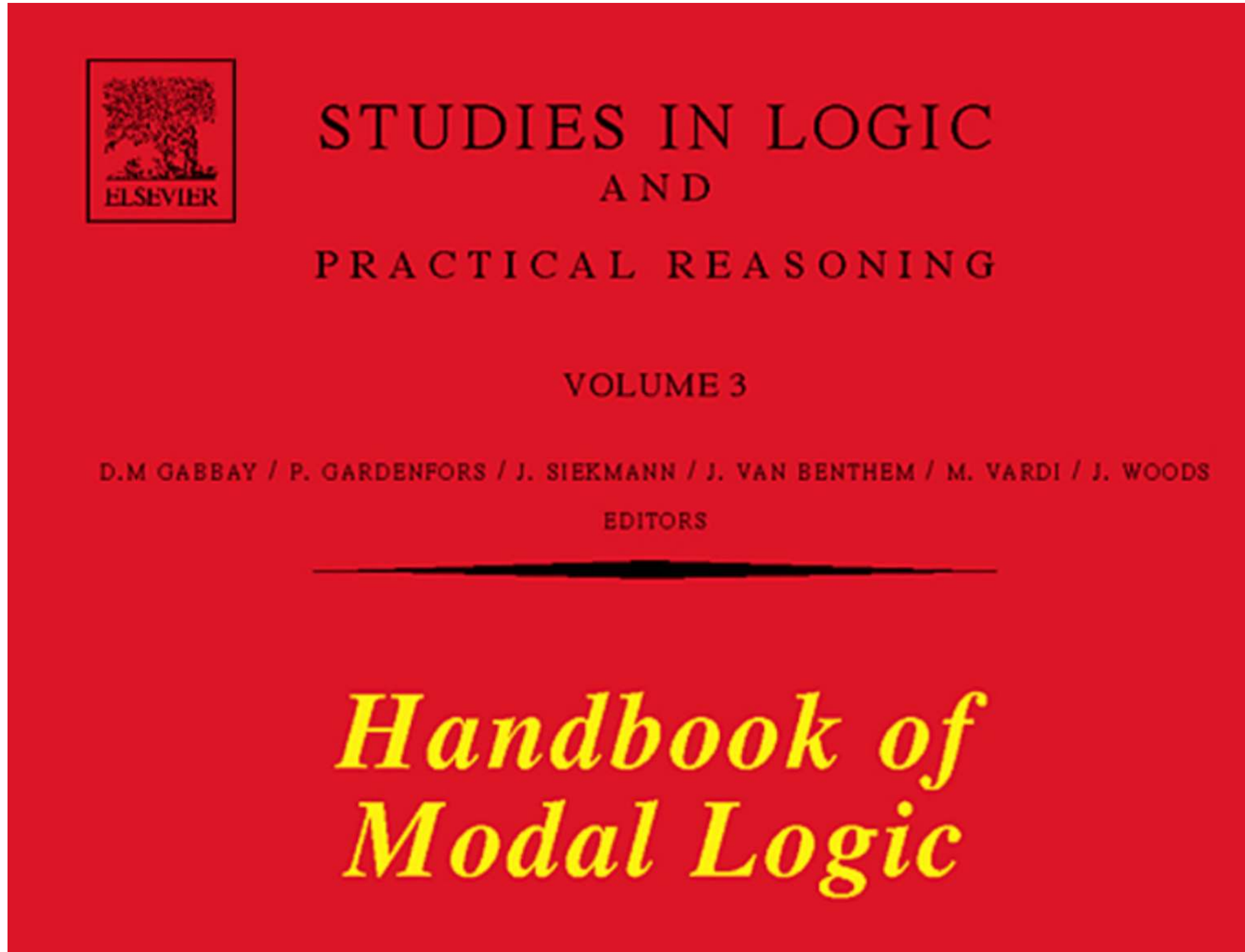
In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Outline

- Motivation
- **HOL** — Classical Higher-Order Logic
 - Non-Classical Logics (why Classical Logic is not enough)
 - **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy** — Flexible KR&R Methodology
- Applications:
 - Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

How to Tame the Logic Zoo?



How to Tame the Logic Zoo?

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 *First steps in relational semantics*

Suppose we have a set of proposition symbols (whose elements we typically write as p, q, r and so on) and a set of modality symbols (whose elements we typically write as m, m', m'' , and so on). The choice of PROP and MOD is called the *signature* (or *similarity type*) of the language; in what follows we'll tacitly assume that PROP is denumerably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle \varphi \mid [m]\varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

How to Tame the Logic Zoo?

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 First steps in relational semantics

Metalinguage

What follows we tacitly assume that PROP is countably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle \varphi \mid [m]\varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

Syntax

How to Tame the Logic Zoo?

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*, *situations*, *worlds* and other things besides. Each R^m in a model is a binary relation on W , and V is a function (the valuation) that assigns to each proposition symbol p in PROP a subset $V(p)$ of W ; think of $V(p)$ as the set of points in $\mathfrak{M}$ where p is true. The first two components $(W, \{R^m\}_{m \in \text{MOD}})$ of $\mathfrak{M}$ are called the *frame* underlying the model. If there is only one relation in the model, we typically write (W, R) for its frame, and (W, R, V) for the model itself. We encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg\varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m w v$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m w v$ we have $\mathfrak{M}, v \models \varphi$.

How to Tame the Logic Zoo?

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*

and $\mathbb{V}$

$V(p)$

$(W, \{$

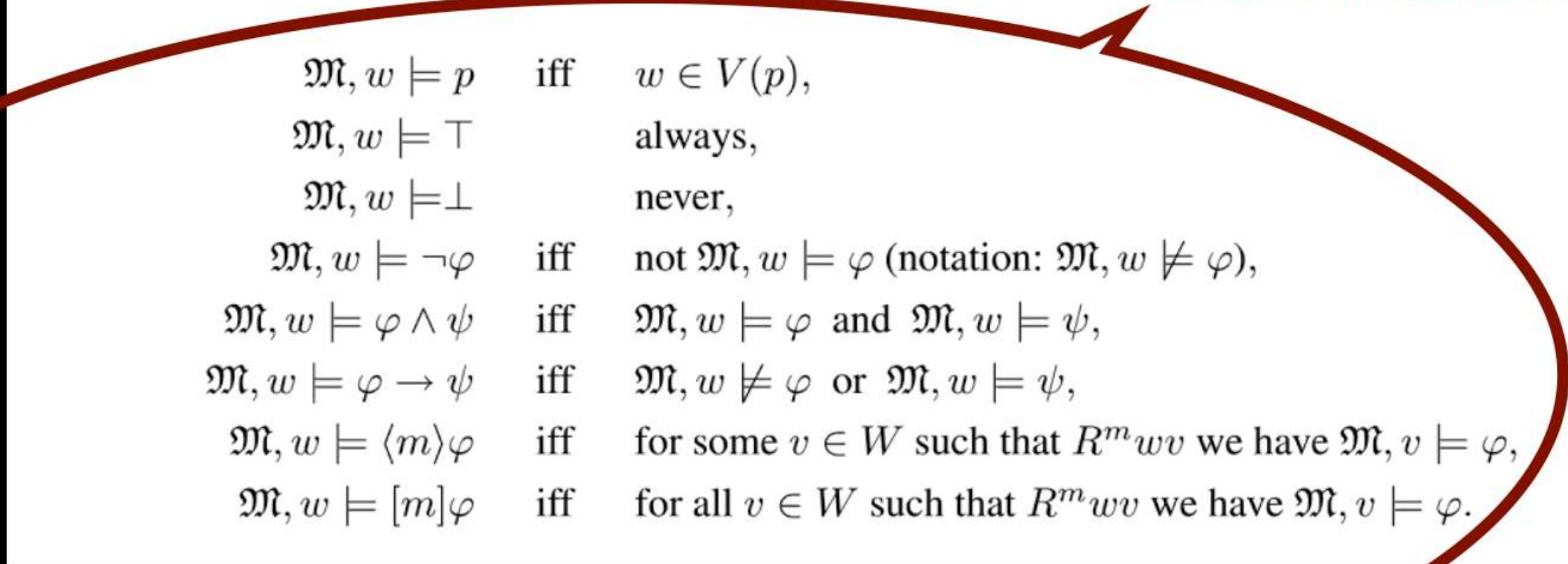
in the

Metalanguage

encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

Semantics



$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg \varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m w v$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m w v$ we have $\mathfrak{M}, v \models \varphi$.

Universal Logical Reasoning

Approach: Shallow Semantic Embedding in HOL

L (target logic): $s, t ::=$ 
HOL (meta-logic): $s, t ::=$ 

Embedding of  **in** 

Signature of HOL (Constants and Logical Symbols):

 $::=$ 
 $::=$ 
 $::=$ 
 $::=$ 

Meta-logical notions:

 $::=$ 

Formulas of **L are directly identified with terms **HOL****

Universal Logical Reasoning

Kripke Style Semantics

(propositional modal logic K)

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Universal Logical Reasoning

Kripke Style Semantics

(propositional modal logic K)

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Standard Translation for Propositional Fragment (encoded in HOL)

- ▶ $P = P_{i \rightarrow o}$
- ▶ $\neg = \lambda \varphi_{i \rightarrow o} \lambda w_i \neg(\varphi w)$
- ▶ $\vee = \lambda \varphi_{i \rightarrow o} \lambda \psi_{i \rightarrow o} \lambda w_i \varphi w \vee \psi w$
- ▶ $\Box = \lambda \varphi_{i \rightarrow o} \lambda w_i \forall v_i R w v \rightarrow \varphi v$

Validity

- ▶ $[\varphi_{i \rightarrow o}] = \forall w_i \varphi w$

Modal Logic (in fact, Hybrid Logic) as a Fragment of HOL

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

Standard Translation extended for Quantifiers (and encoded in HOL)

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$



Kurt Gödel



Dana Scott

Ontologische Beweise Feb 10, 1970

$P(\varphi)$ φ is positive (i.e. $\varphi \in P$)

At 1 $P(\varphi), P(\psi) \supset P(\varphi \wedge \psi)$ At 2 $P(\varphi) \supset P(\neg \neg \varphi)$

[1 $G(x) \equiv (\varphi) [P(\varphi) \supset \varphi(x)]$ (God)

[2 $\varphi \text{ Ess } x \equiv (\psi) [\psi(x) \supset N(\exists y) [\varphi(y) \supset \psi(y)]]$ (Essence of x)

$p \supset_N q = N(p \supset q)$ Necessity

At 2 $P(\varphi) \supset N P(\varphi)$
 $\neg P(\varphi) \supset N \neg P(\varphi)$ } Because it follows from the nature of the property

Th. $G(x) \supset G \text{ Ess } x$

Df. $E(x) \equiv (\varphi) [\varphi \text{ Ess } x \supset N \exists x \varphi(x)]$ necessary Existence

Ax 3 $P(E)$

Th. $G(x) \supset N(\exists y) G(y)$
 thus $(\exists x) G(x) \supset N(\exists y) G(y)$
 $M(\exists x) G(x) \supset M N(\exists y) G(y)$
 $" \supset N(\exists y) G(y)$ M = possibility

any two instances of x are nec. equivalent,
 exclusive or " and for any number of humanoids

$M(\exists x) G(x)$ means ^{the system of} all pos. props. is com-
 patible This is true because of:

At 4: $P(\varphi), \varphi \supset_N \psi \supset P(\psi)$ which implies

~~since~~ $\begin{cases} x=x & \text{is positive} \\ x \neq x & \text{is negative} \end{cases}$

But if a system S of pos. props. were inconsistent it would mean that the same prop. is (which is positive) would be $x \neq x$

Positive means positive in the modal arith-
 sense (independently of the accidental structure of
 the world). (Only ^{pure} in the ar. sense. It in-
 cludes ^{pure} "attribution" as opposed to "privation"
 (or containing privation) - This requires no other proof

$\exists / \neg \varphi$ privation: $(x) N \neg \varphi(x)$ (Contradiction: $\varphi(x) \supset x \neq x$
 hence $x \neq x$ positive, not $x = x$ negative. At-
 tribution of pos. prop. to the existence of pos. prop.

diag
 x i.e. the normal form in terms of elem. prop. contains a
 member without negation.



Anthony C. Anderson



Melvin Fitting

Kurt Gödel's Ontological Argument (in higher-order modal logic)

Universal Logical Reasoning in HOL

```
1 theory QML_S5U imports Main
2 begin
3   typedecl i (* "type for possible worlds" *)
4   typedecl μ (* "type for individuals" *)
5   type_synonym σ = "(i⇒bool)"
6
7   abbreviation mnot ("¬" [52]53) where "¬φ ≡ λw. ¬φ(w)"
8   abbreviation mnotpred ("¬_" [52]53) where "¬_φ ≡ λx.λw. ¬φ(x)(w)"
9   abbreviation mand (infixr "∧" 51) where "φ ∧ ψ ≡ λw. φ(w) ∧ ψ(w)"
10  abbreviation mor (infixr "∨" 50) where "φ ∨ ψ ≡ λw. φ(w) ∨ ψ(w)"
11  abbreviation mimp (infixr "→" 49) where "φ → ψ ≡ λw. φ(w) → ψ(w)"
12  abbreviation mequ (infixr "≡" 48) where "φ ≡ ψ ≡ λw. φ(w) ↔ ψ(w)"
13  abbreviation mall ("∀") where "∀φ ≡ λw.∀x. φ(x)(w)"
14  abbreviation mallB (binder "∀" [8]9) where "∀x. φ(x) ≡ ∀φ"
15  abbreviation mexi ("∃") where "∃φ ≡ λw.∃x. φ(x)(w)"
16  abbreviation mexiB (binder "∃" [8]9) where "∃x. φ(x) ≡ ∃φ"
17  abbreviation mbox ("□") where "□φ ≡ λw.∀v. φ(v)"
18  abbreviation mdia ("◇") where "◇φ ≡ λw.∃v. φ(v)"
19
20  abbreviation valid ("□_" [7]8) where "□_p ≡ ∀w. p w"
21
22  lemma "□φ → φ" sledgehammer by simp
23  lemma "□φ → ◇φ" sledgehammer by simp
24  lemma "□φ → □(□φ)" sledgehammer by simp
25 end
```

```
1 theory Scott_S5U imports QML_S5U
2 begin
3   consts P :: "(μ⇒σ)⇒σ" (* Positive *)
4   axiomatization where
5     A1a: "[∀φ. P(¬φ) → ¬P(φ)]" and
6     A1b: "[∀φ. ¬P(φ) → P(¬φ)]" and
7     A2: "[∀φ ψ. P(φ) ∧ □(∀x. φ(x) → ψ(x)) → P(ψ)]"
8   definition G where "G(x) = (∀φ. P(φ) → φ(x))" (* Def. of God *)
9   axiomatization where
10    A3: "[P(G)]" and
11    A4: "[∀φ. P(φ) → □(P(φ))]"
12  definition ess (infixr "ess" 85) where
13    "φ ess x = φ(x) ∧ (∀ψ. ψ(x) → □(∀y. φ(y) → ψ(y)))" (* Essence *)
14  definition NE where "NE(x) = (∀φ. φ ess x → □(∃ φ))" (* Nec. Existence *)
15  axiomatization where
16    A5: "[P(NE)]"
17
18  theorem T3: "[□(∃x. G(x))]" (* Necessarily, God ∃ *)
19  sledgehammer [provers = remote_leo2, verbose] (* LE0-II proof in 2,5sec *)
20  by (metis (lifting, full_types)
21      A1a A1b A2 A3 A4 A5 G_def NE_def ess_def)
22 end
```

Higher-order modal logic $S5^U$ in HOL

Ontological argument in $S5^U$

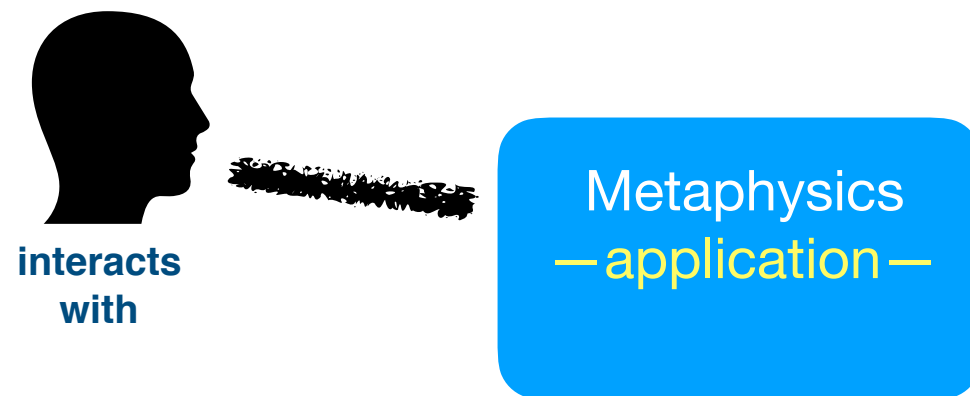
With a few lines of formal HOL code:

- ▶ novel applications (here in metaphysics)
- ▶ excellent degree of automation (automation of Gödel's argument)
- ▶ while still being intuitive (readable by philosophers)

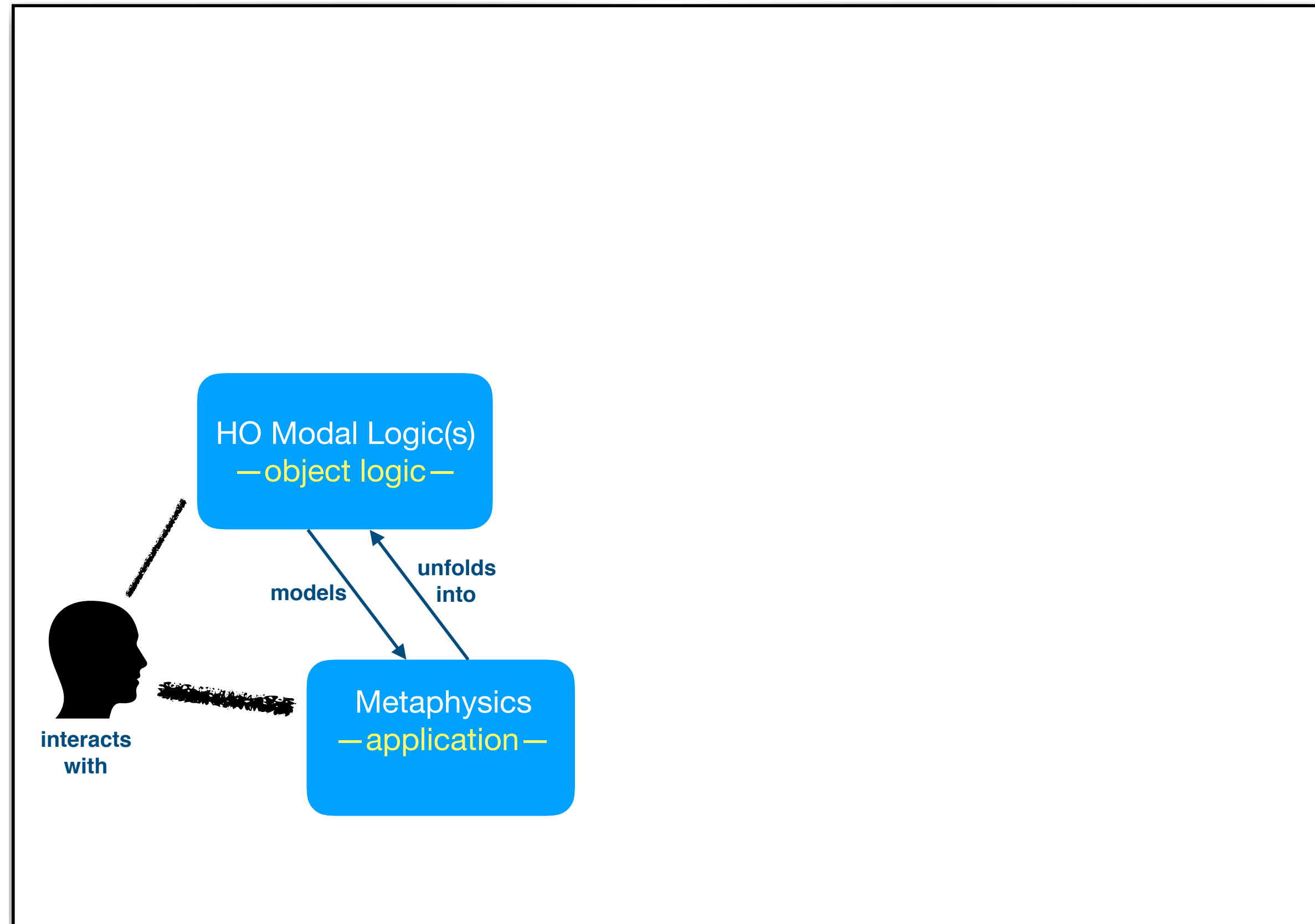
Further reading:

[ECAI 2014, IJCAI 2016, Open Philosophy 2019]

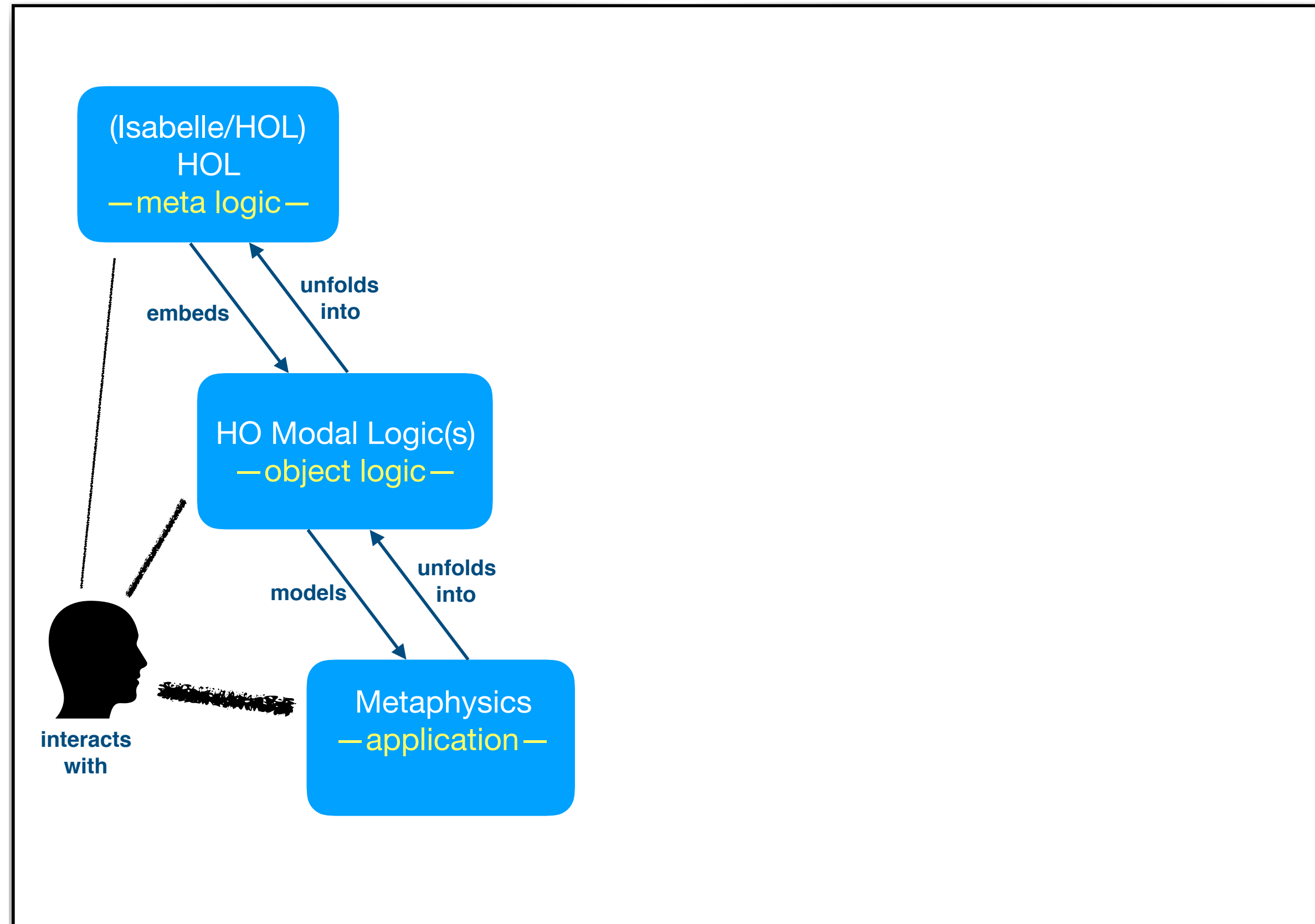
Universal Logical Reasoning in HOL



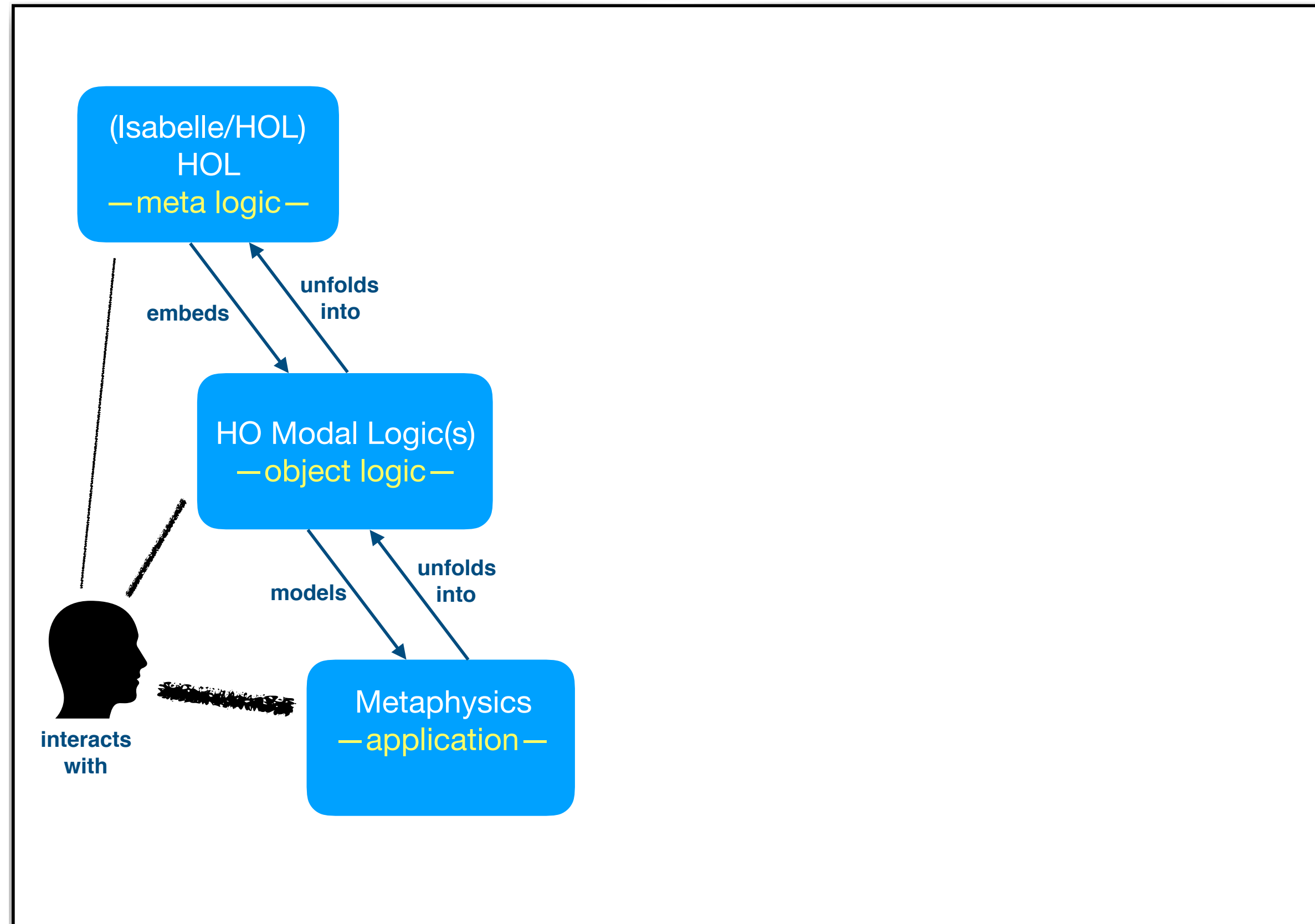
Universal Logical Reasoning in HOL



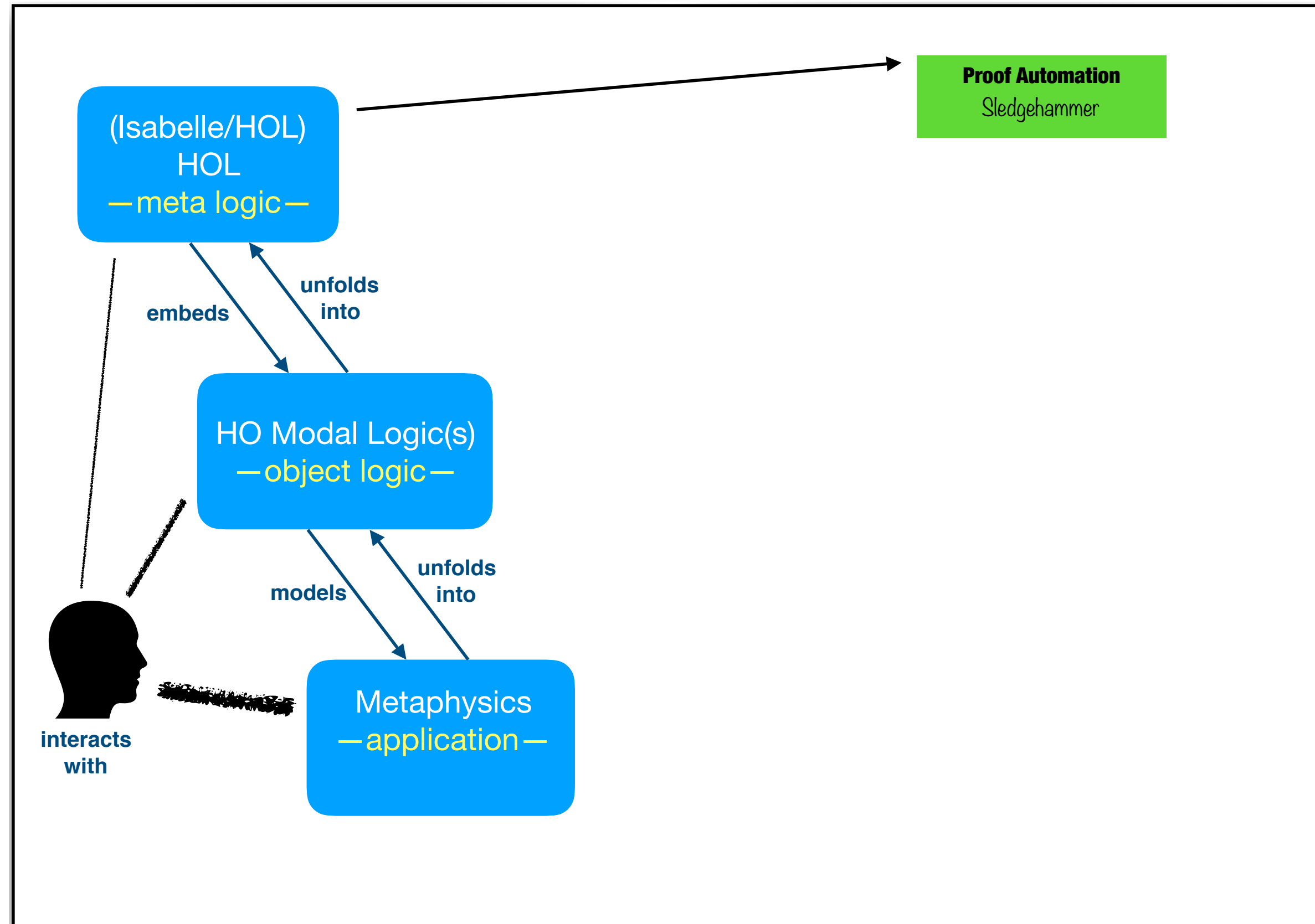
Universal Logical Reasoning in HOL



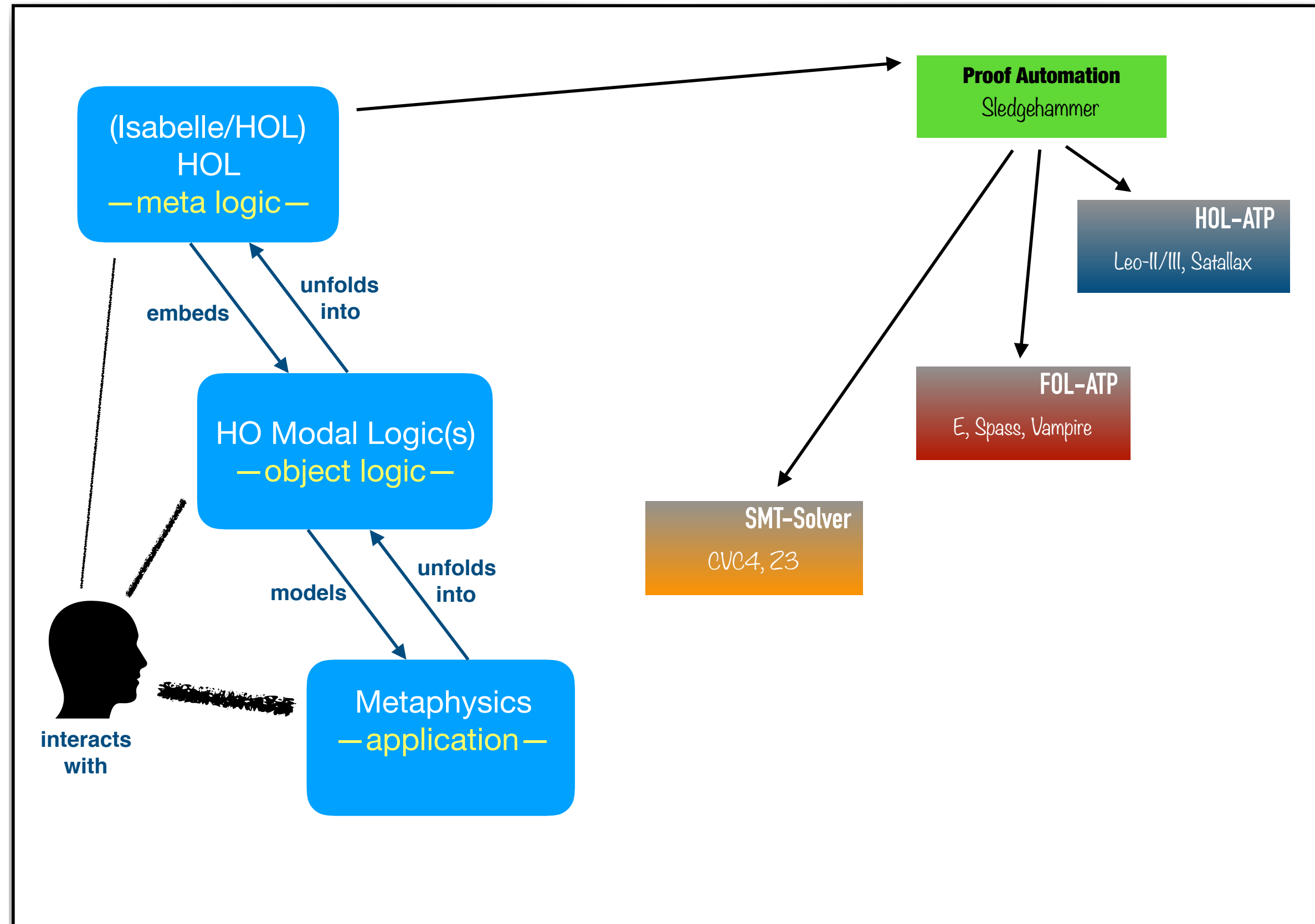
Universal Logical Reasoning in HOL



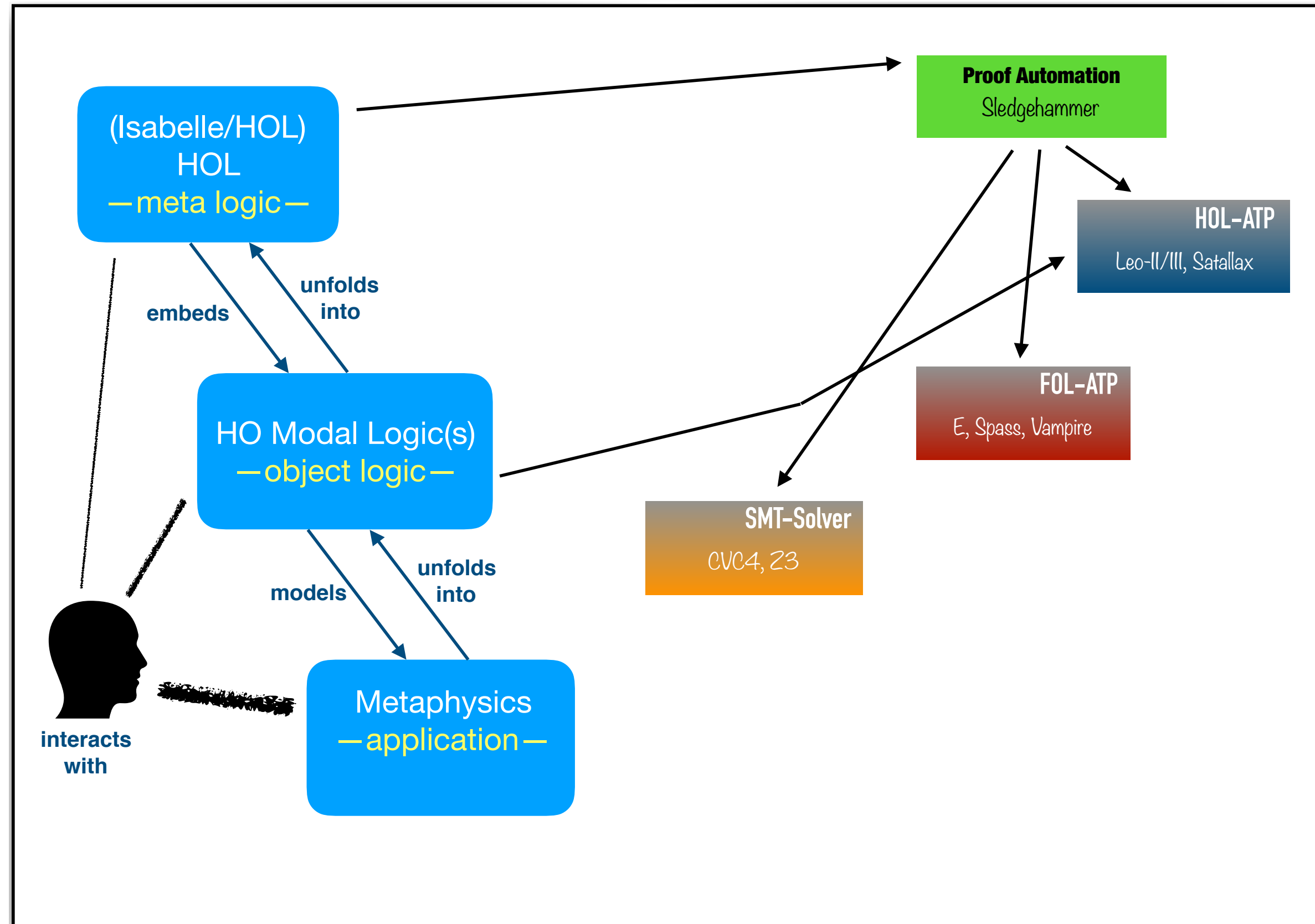
Universal Logical Reasoning in HOL



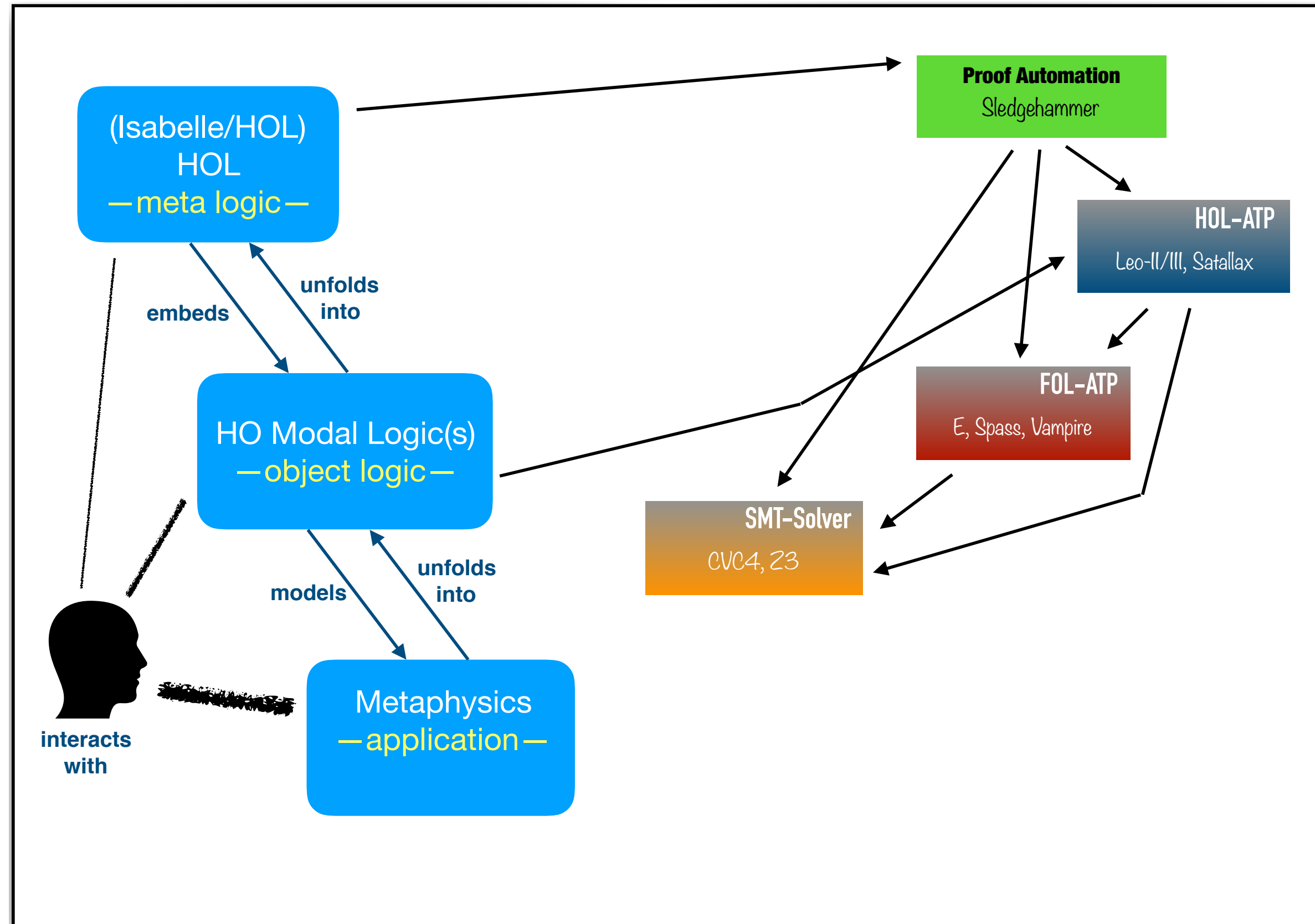
Universal Logical Reasoning in HOL



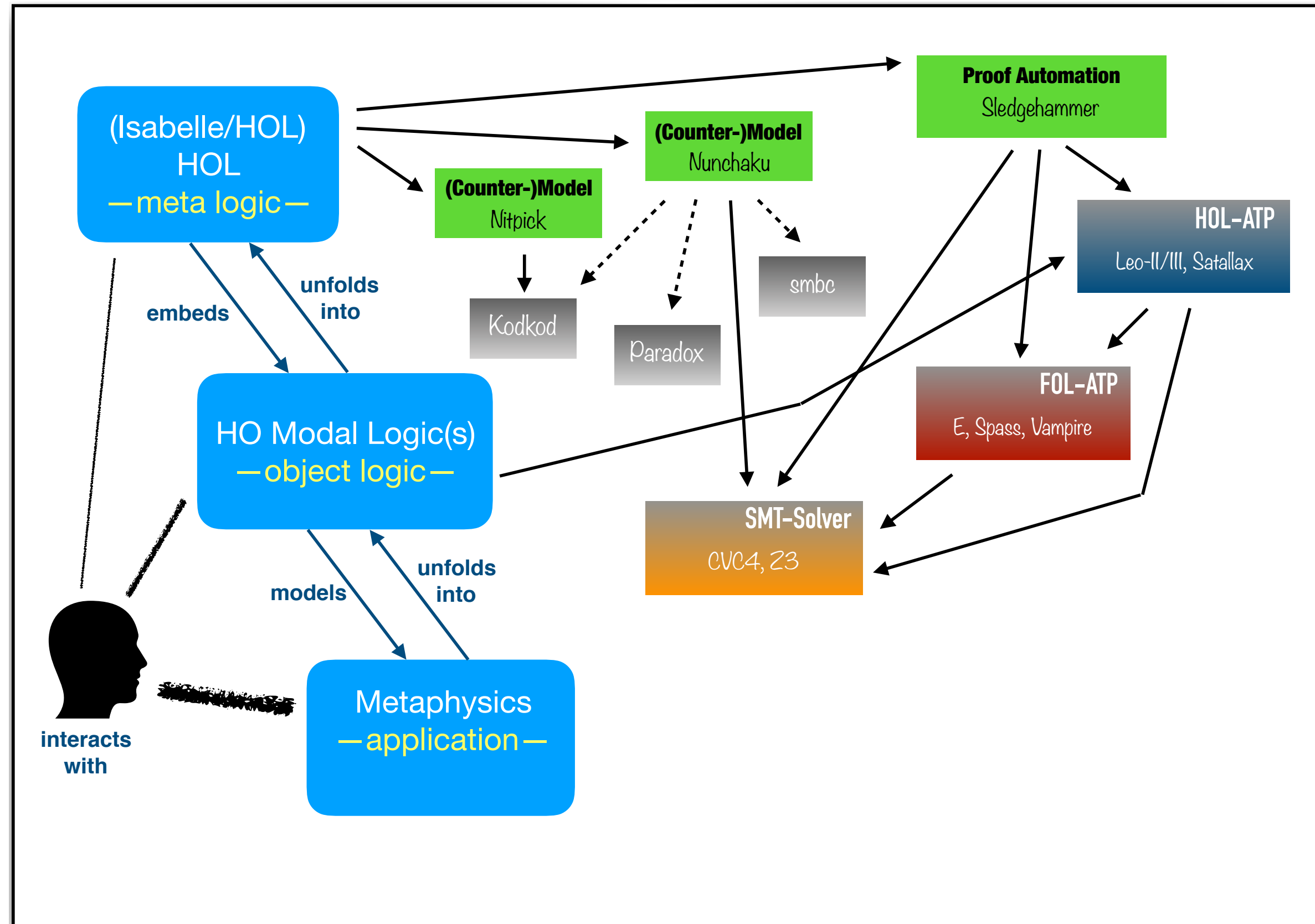
Universal Logical Reasoning in HOL



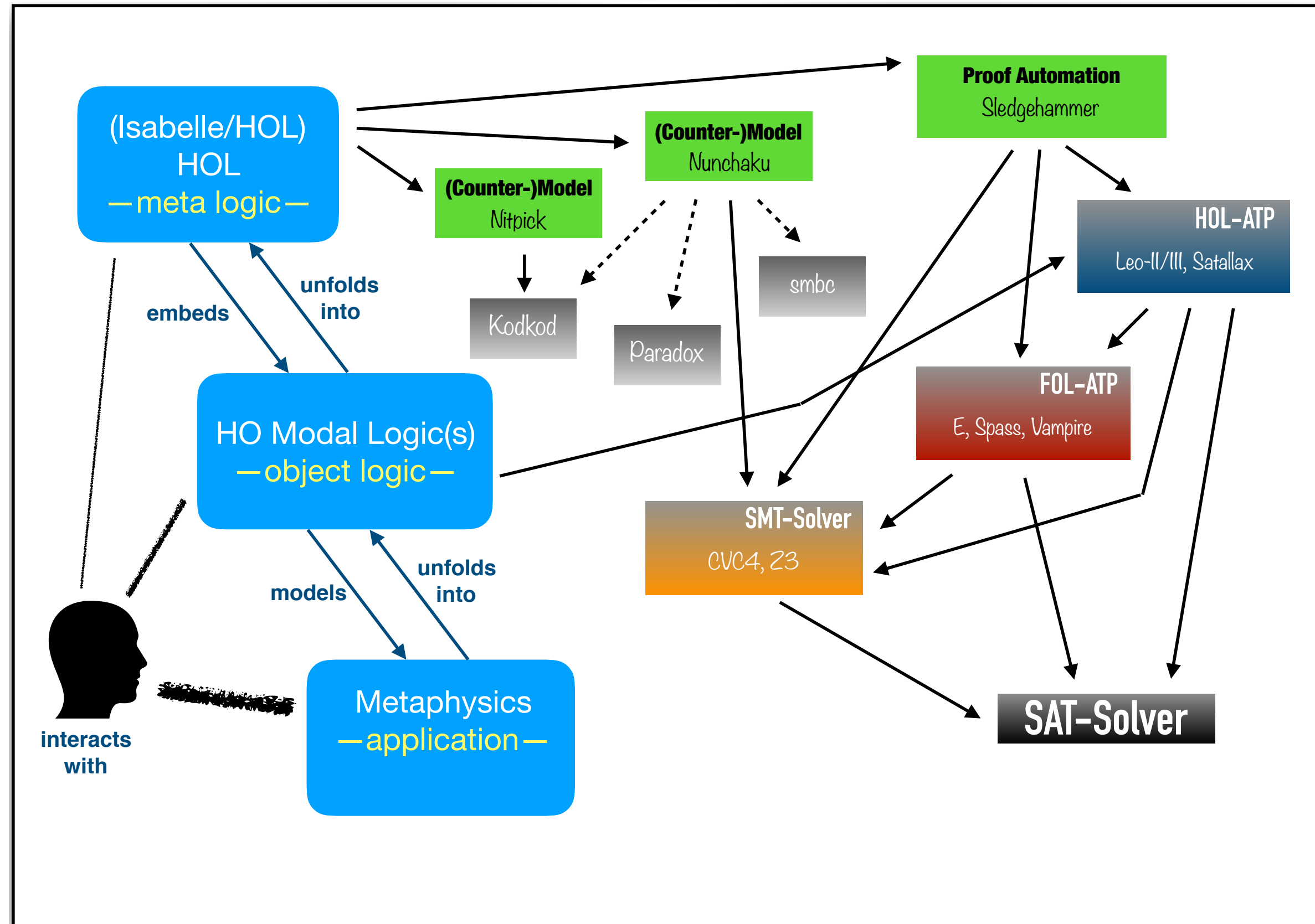
Universal Logical Reasoning in HOL



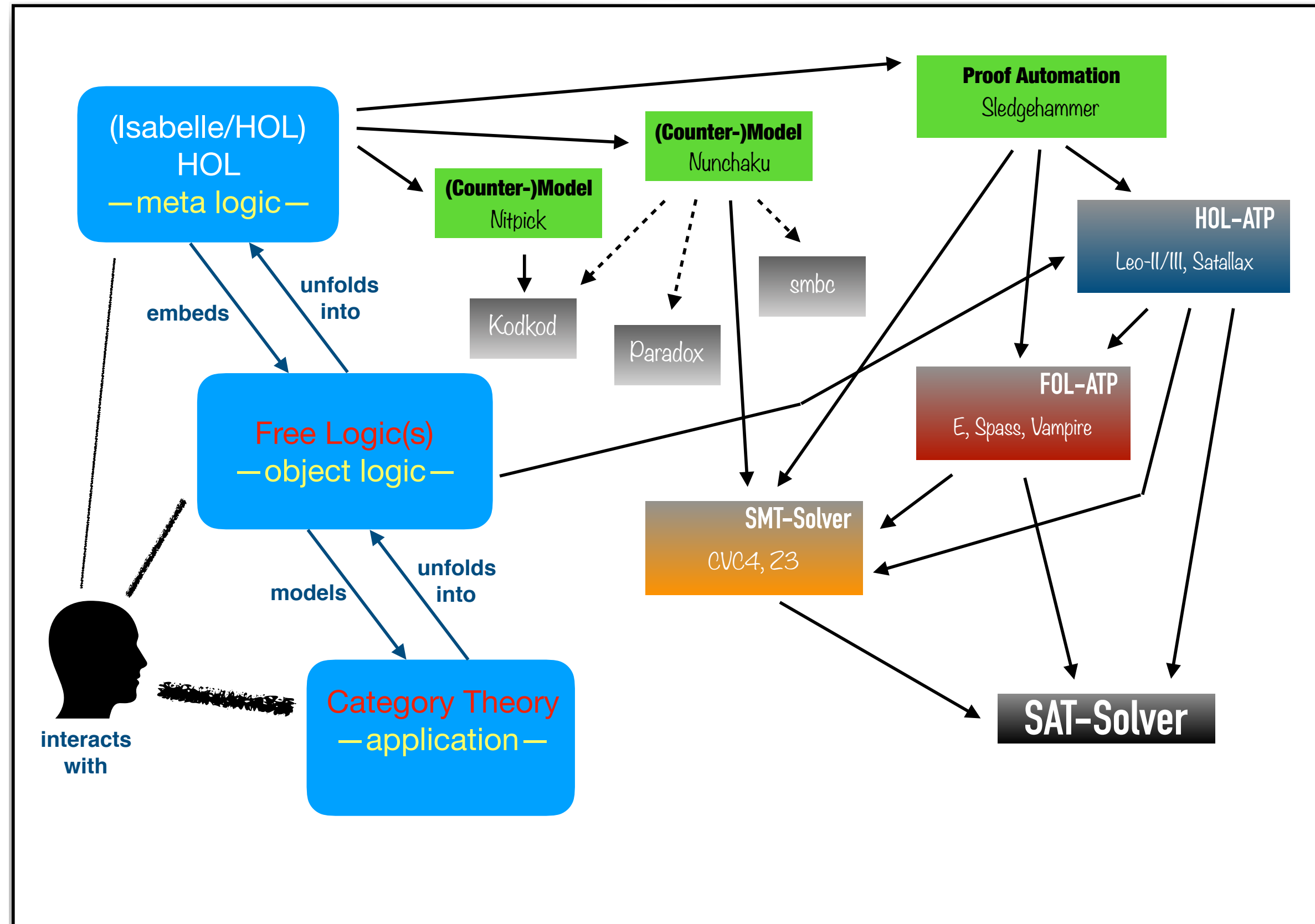
Universal Logical Reasoning in HOL



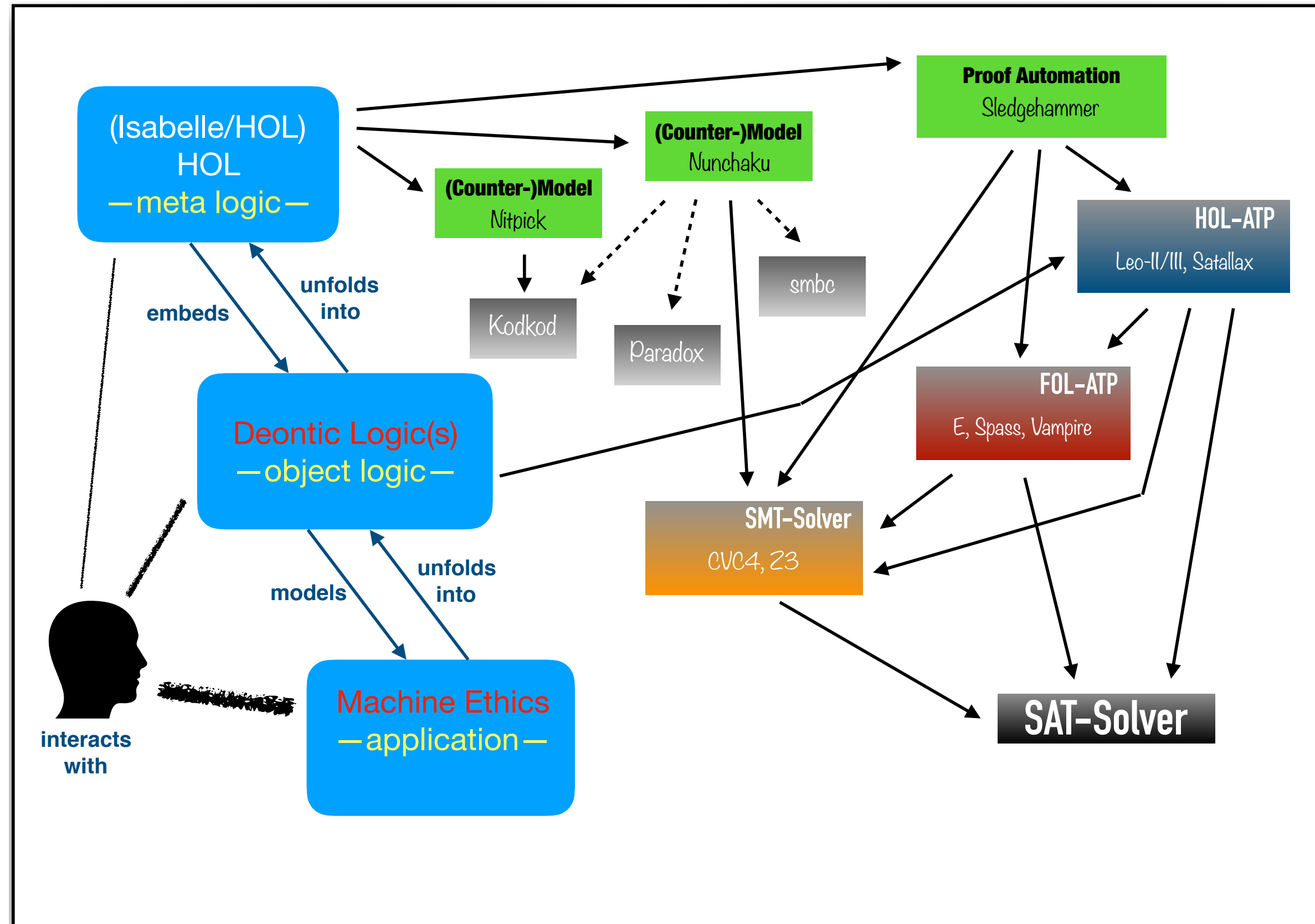
Universal Logical Reasoning in HOL



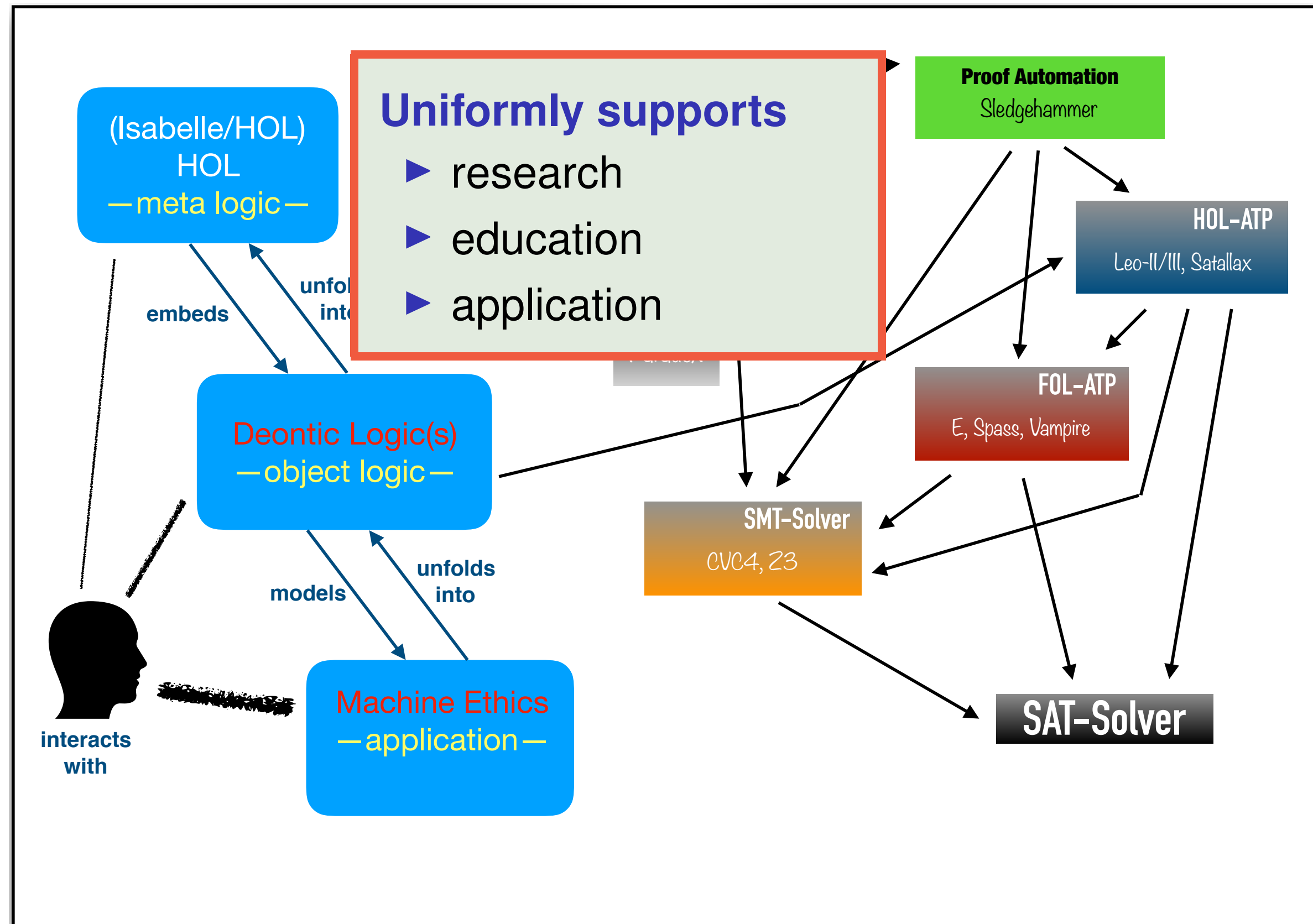
Universal Logical Reasoning in HOL



Universal Logical Reasoning in HOL



Universal Logical Reasoning in HOL



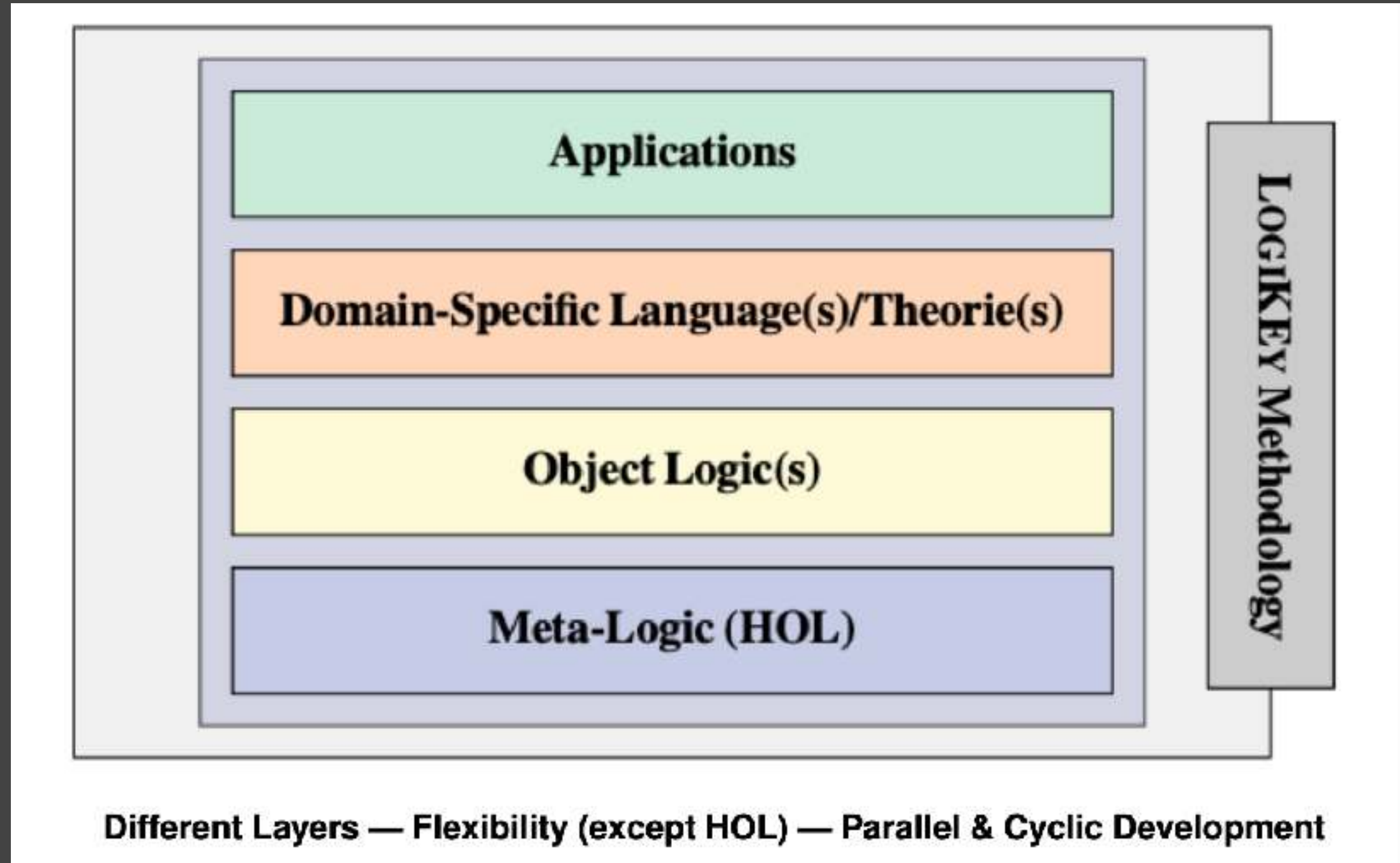
Outline

- Motivation
- **HOL** — Classical Higher-Order Logic
 - Non-Classical Logics (why Classical Logic is not enough)
- **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy — Flexible KR&R Methodology**
- Applications:
 - Rational Arguments: Metaphysics/Philosophy
 - Foundations: Maths, Philosophy, Sciences
 - Normative Reasoning: AI and Ethics & Law

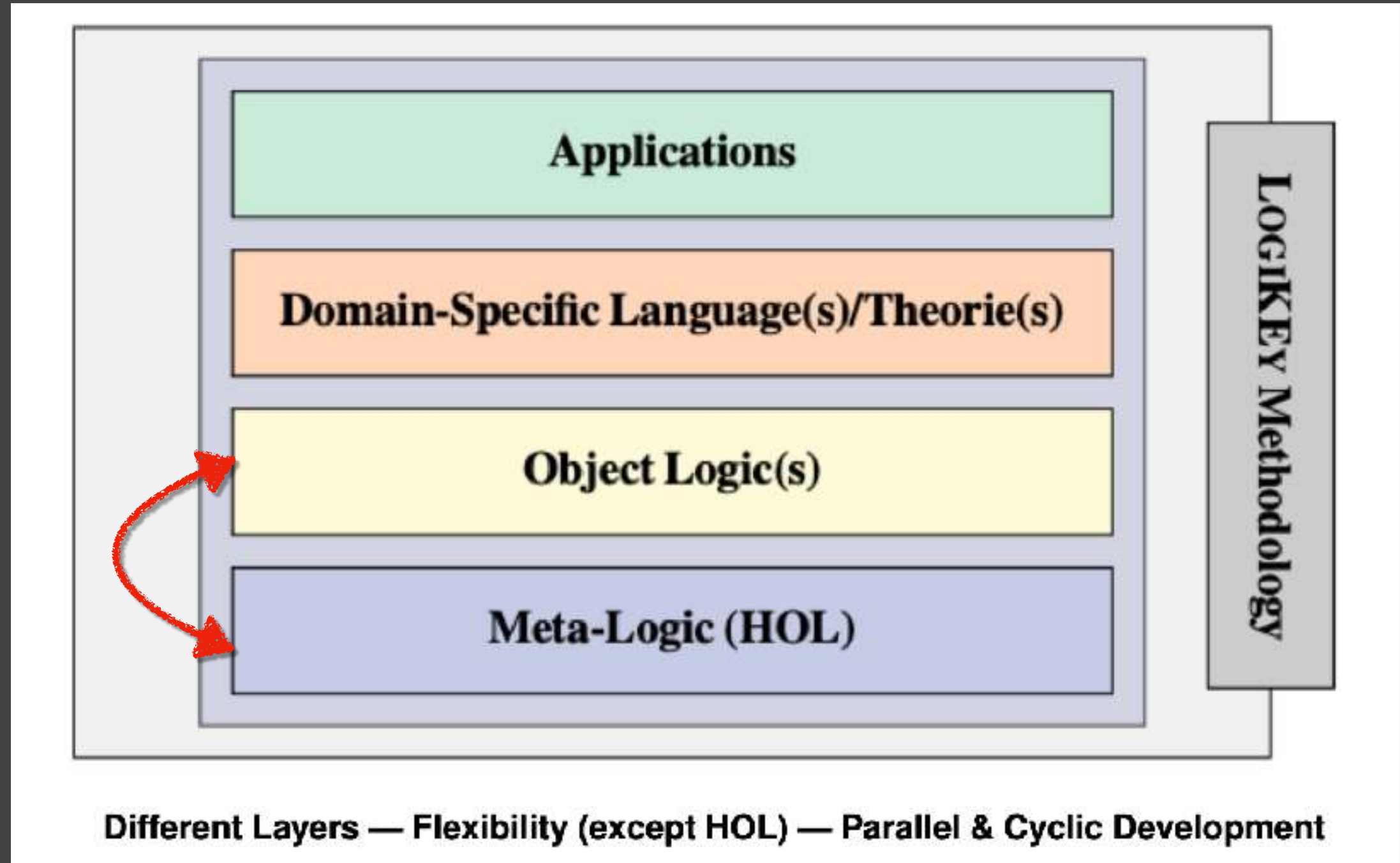
Logico-Pluralistic KR&R Methodology: LogiKEy



jww
colleagues at
U Luxembourg



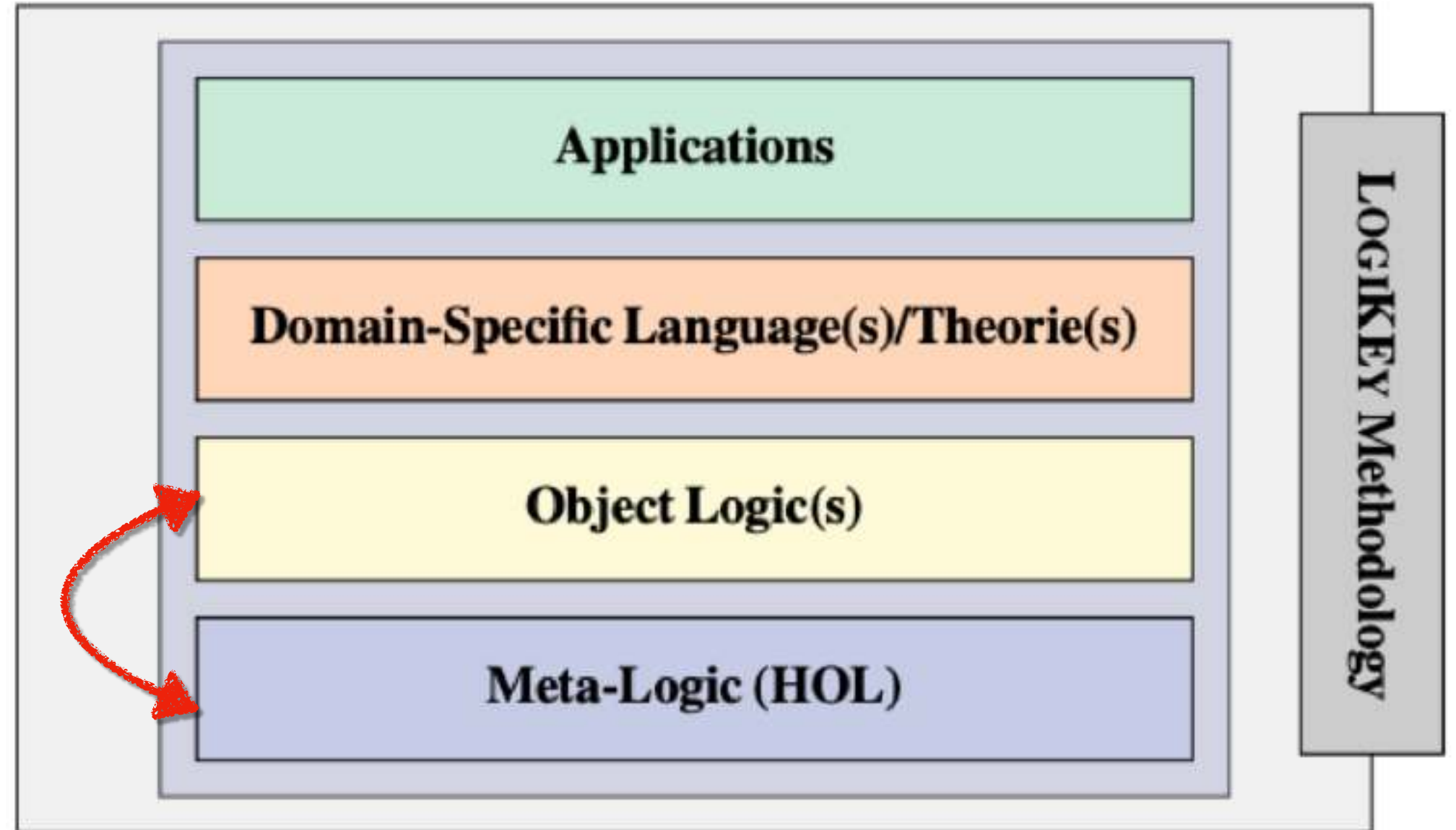
Universal (Meta-)Logical Reasoning



Universal (Meta-)Logical Reasoning



...
developed from
...



Different Layers — Flexibility (except HOL) — Parallel & Cyclic Development

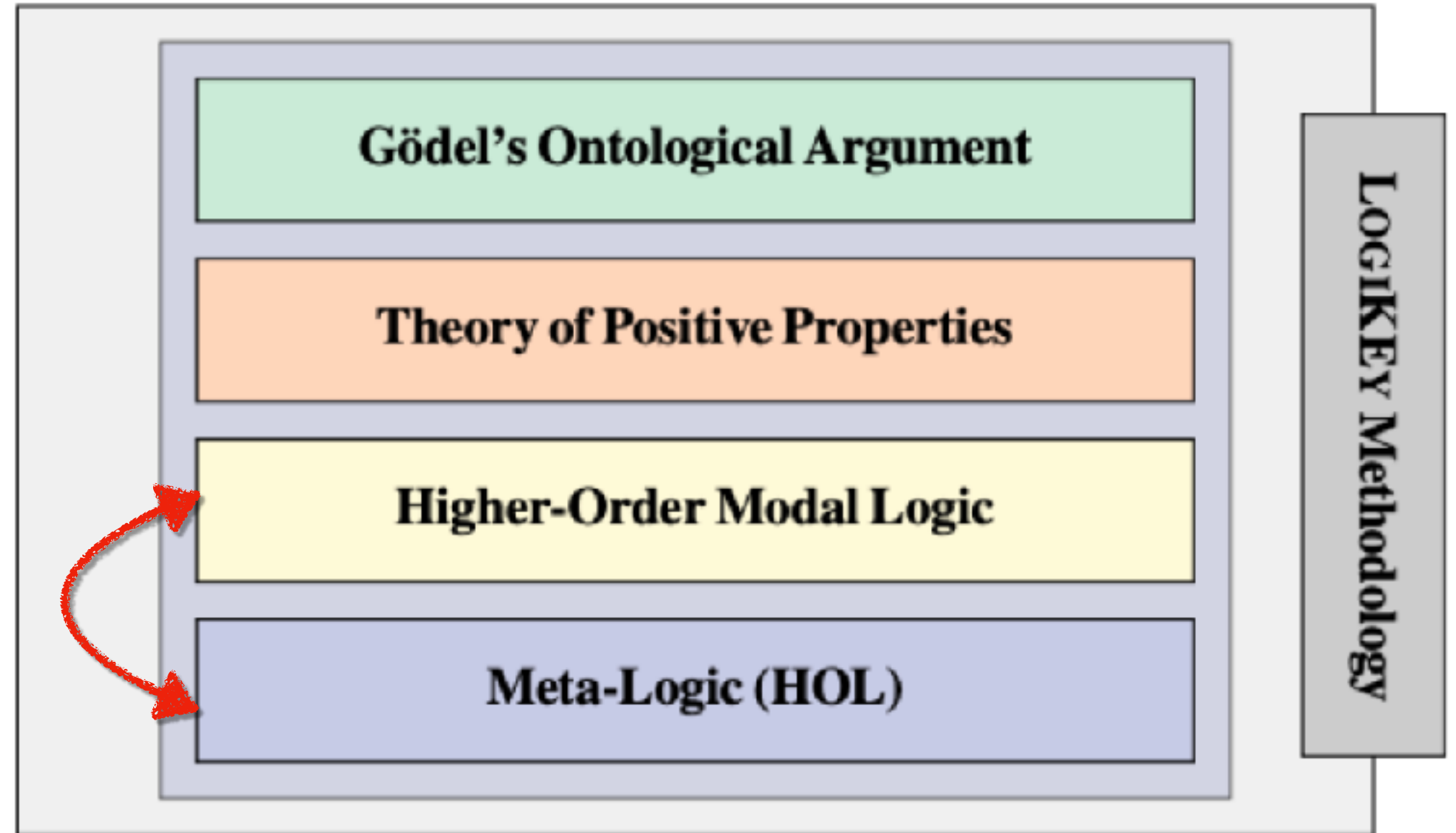
Universal (Meta-)Logical Reasoning

Metaphysics



Automating Gödel's Ontological Proof of God's Existence with Higher-order Automated Theorem Provers

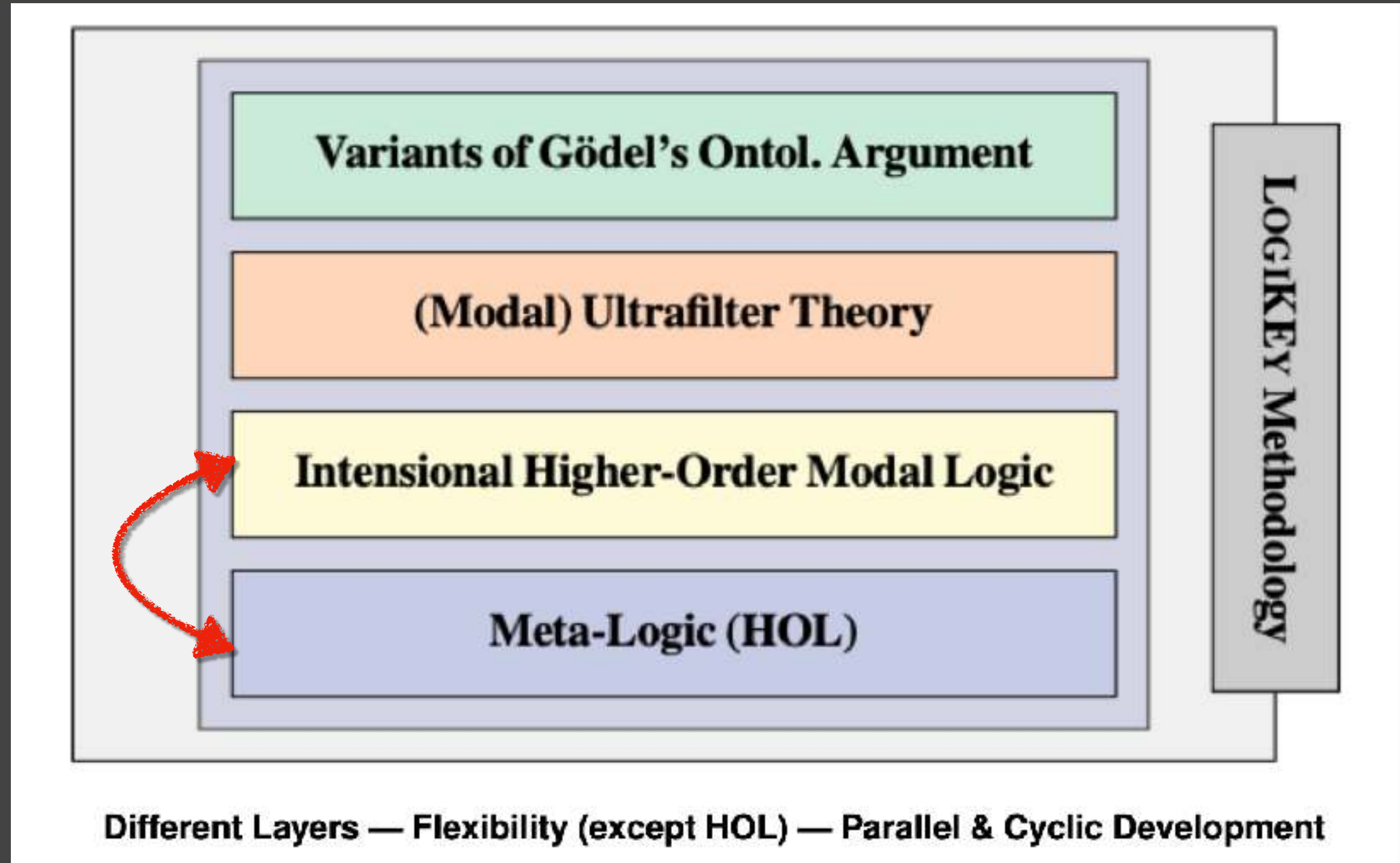
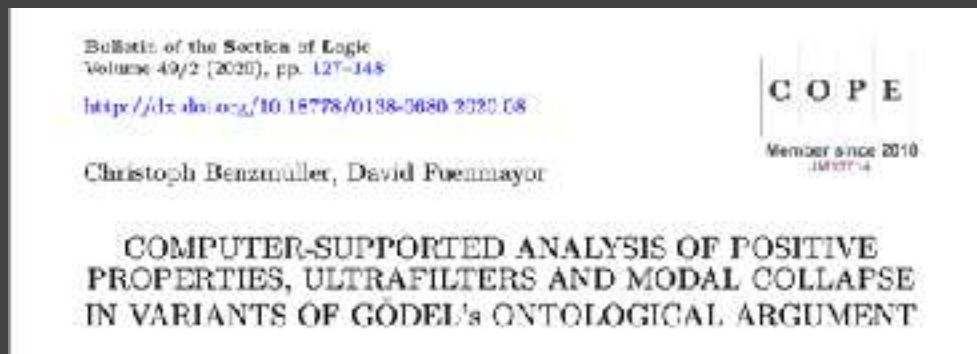
Authors: Christoph Benzmüller, Bruno Woltzenlogel Paleo
Pages: 93 - 98
DOI: 10.3233/978-1-61499-419-0-93
Series: Frontiers in Artificial Intelligence and Applications
Ebook: Volume 263, ECAI 2014



Different Layers — Flexibility (except HOL) — Parallel & Cyclic Development

Universal (Meta-)Logical Reasoning

Metaphysics



Universal (Meta-)Logical Reasoning

Law & Ethics

License:  Creative Commons Attribution 4.0 license (CC BY 4.0)
when quoting this document, please refer to the following
DOI: [10.4230/LIPics.ITP.2021.7](https://doi.org/10.4230/LIPics.ITP.2021.7)
URN: [urn:nbn:de:0030-drops-139028](https://nbn-resolving.org/urn:nbn:de:0030-drops-139028)
URL: <https://drops.dagstuhl.de/opus/volltexte/2021/13902/>

Benzmüller, Christoph ; Fuenmayor, David

Value-Oriented Legal Argumentation in Isabelle/HOL

pdf-format: [LIPics-ITP-2021-7.pdf \(2 MB\)](#)

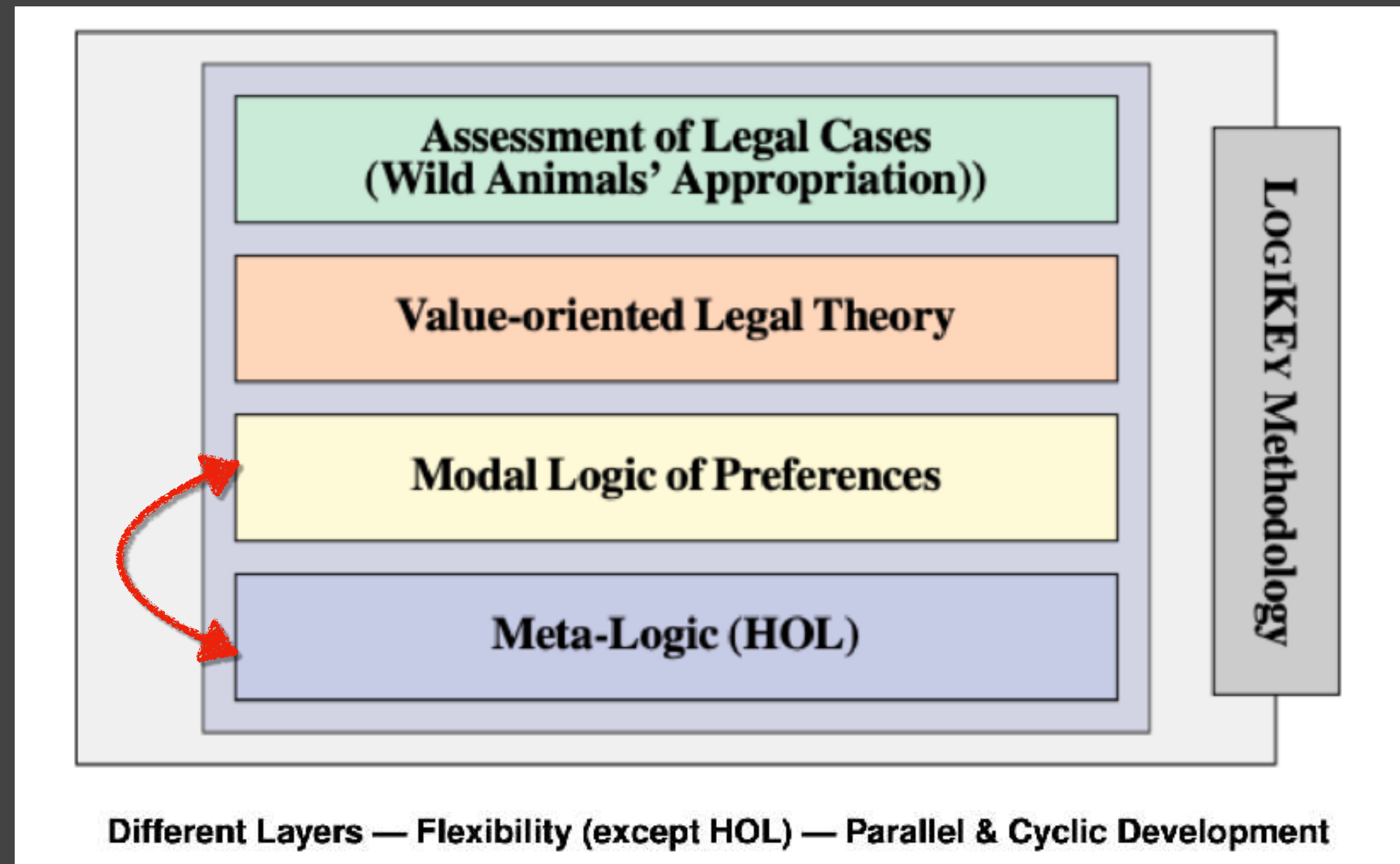
 [arXiv > cs > arXiv:2006.12789](https://arxiv.org/abs/2006.12789)

Computer Science > Artificial Intelligence

(Submitted on 23 Jun 2020 (v1); last revised 30 Mar 2022 (this version, v5))

Modelling Value-oriented Legal Reasoning in LogiKey

Christoph Benzmüller, David Fuenmayor, Bertram Lomfeld



Universal (Meta-)Logical Reasoning

Category Theory

 International Congress on Mathematical Software
→ ICMS 2016: *Mathematical Software – ICMS 2016* pp 43–50 | Cite as

Automating Free Logic in Isabelle/HOL

Christoph Benz Müller  & Dana Scott

 Springer Link

Published: 01 January 2019

Automating Free Logic in HOL, with an Experimental Application in Category Theory

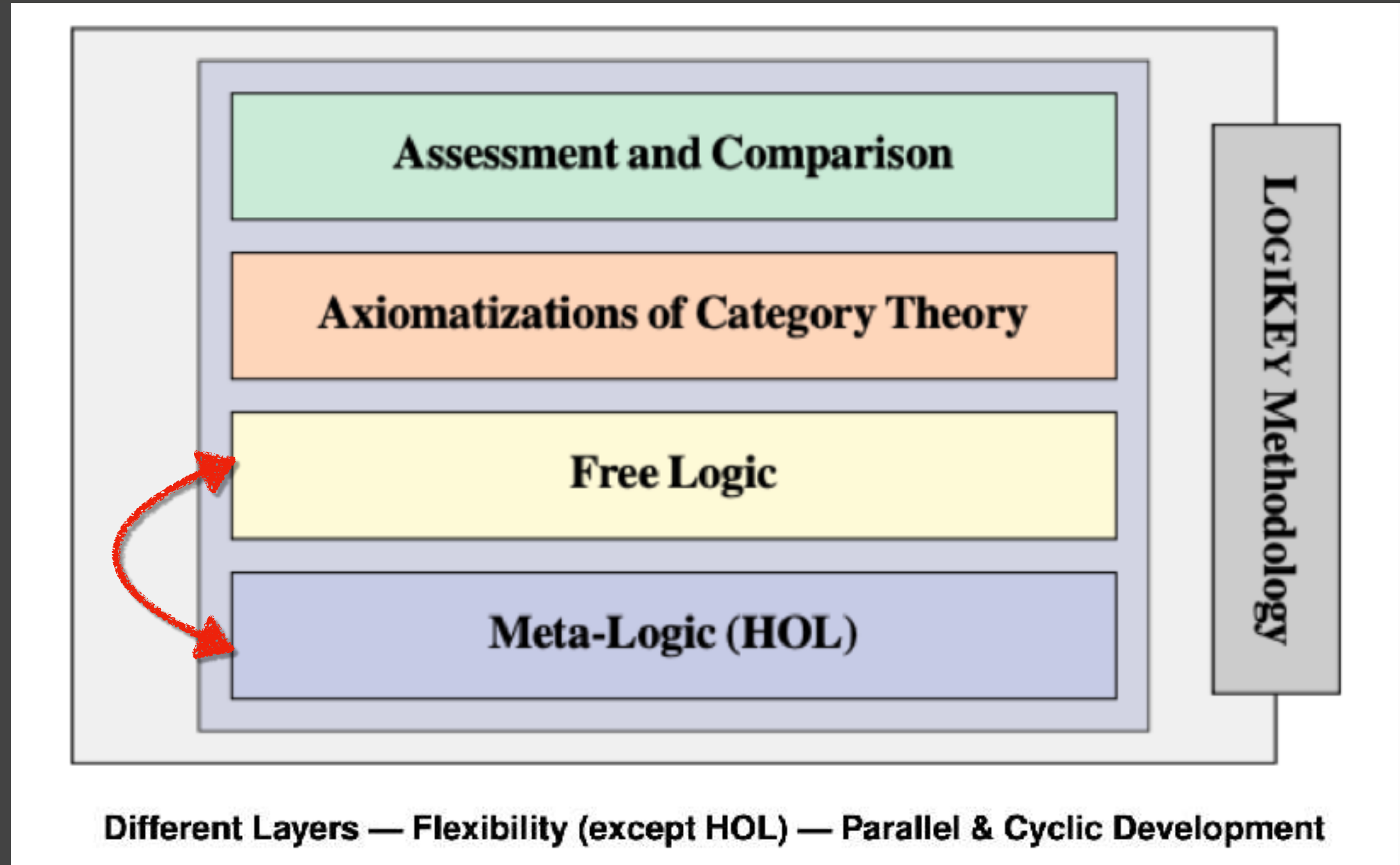
Christoph Benz Müller  & Dana S. Scott

Journal of Automated Reasoning **64**, 53–72 (2020) | Cite this article

 International Conference on Relational and Algebraic Methods in Computer Science
→ RAMICS 2020: *Relational and Algebraic Methods in Computer Science* pp 302–317

Computer-Supported Exploration of a Categorical Axiomatization of Modeloids

Lucca Tiemens , Dana S. Scott, Christoph Benz Müller & Miroslav Bena



AI & Representing Objects

Abstract representations: objects of study in AI from the very beginning. Bibel calls them **Representing Objects (ROBs)**.

- ROBs are key to symbolic AI
- ROBs can be experimented with in the computer
- ROBs thus become physical/accessible (part of nature?)

Symbolic AI & logic in combination with computer experimentation on ROBs deserve increased attention as experimental (natural?) science.

Examples: mathematical theories, metaphysical theories, legal theories, etc., including even their **underlying logics**



Wolfgang Bibel
(German AI Pioneer)

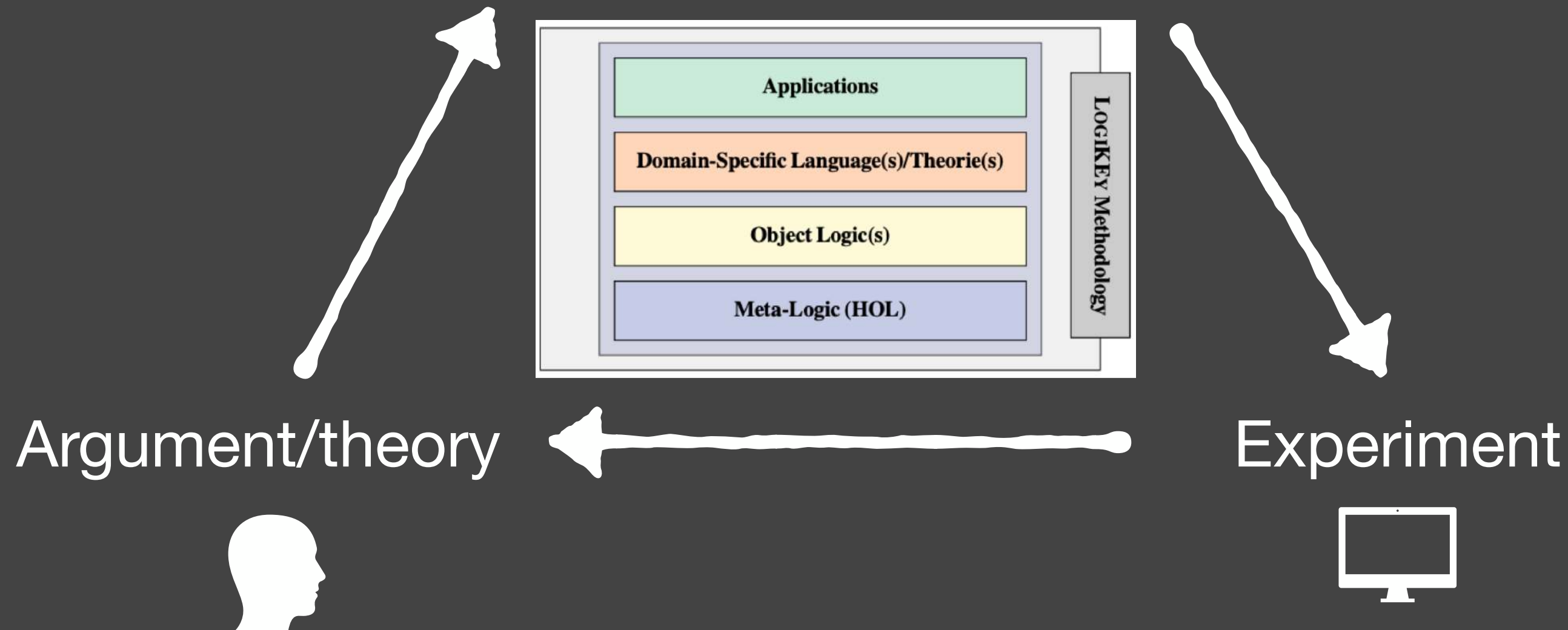


AI & Representing Objects

Human-Computer
Interaction



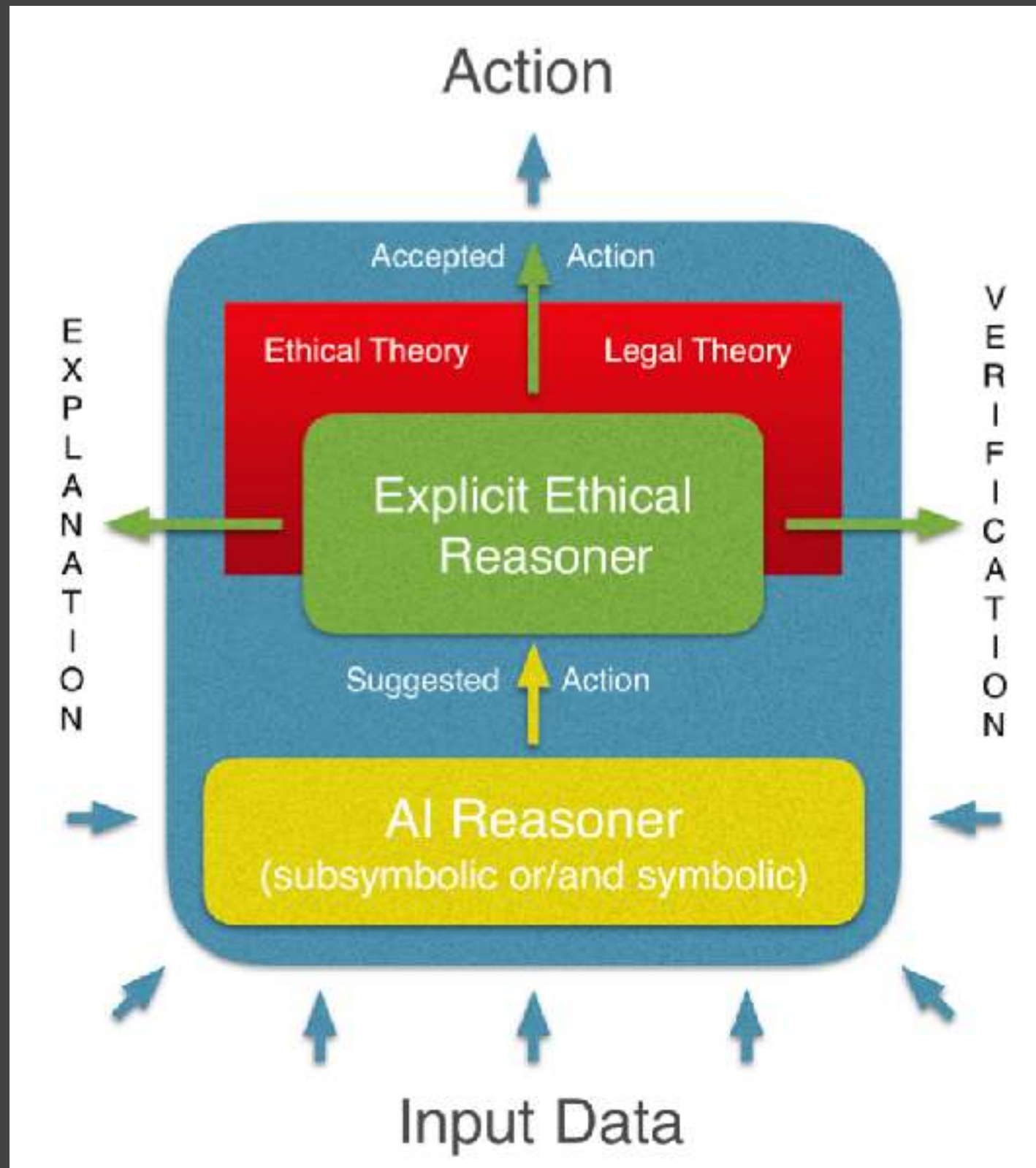
Representing object (logical representation)



Outline

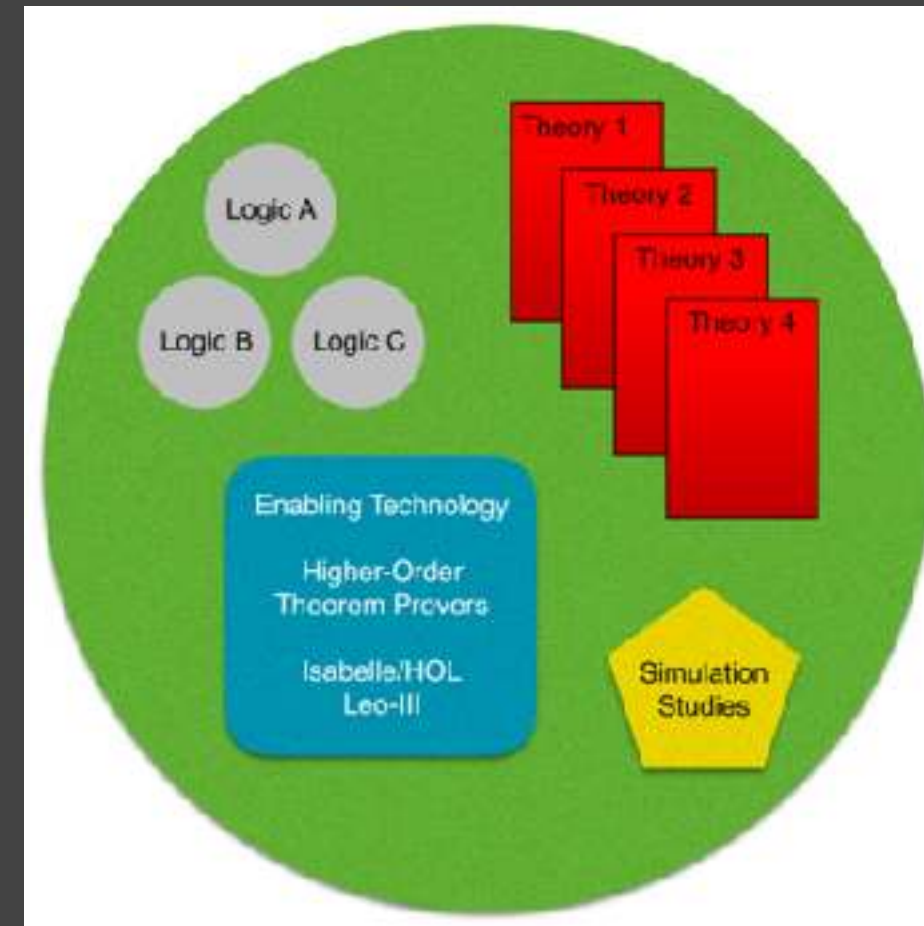
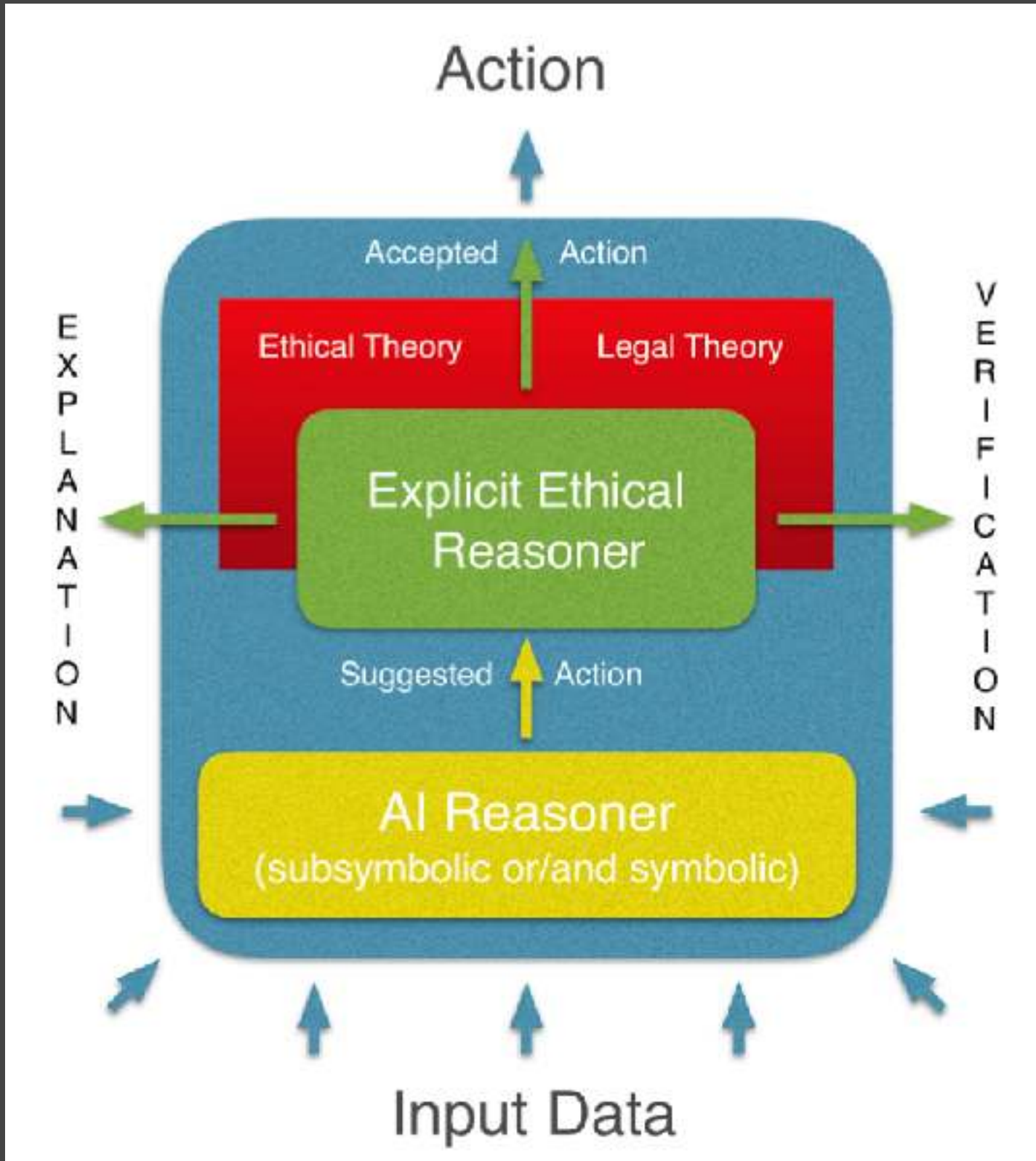
- Motivation
- **HOL** — Classical Higher-Order Logic
 - Non-Classical Logics (why Classical Logic is not enough)
- **Universal Meta-Logical Reasoning** (why Classical Logic is enough)
- **LogiKEy** — Flexible KR&R Methodology
- Applications:
 - Rational Arguments, e.g. in Metaphysics
 - Foundations of Mathematics and Philosophy
 - Normative Reasoning: AI and Ethics & Law

ETHICO-LEGAL GOVERNORS?



jww
colleagues at
U Luxembourg

ETHICO-LEGAL GOVERNORS?

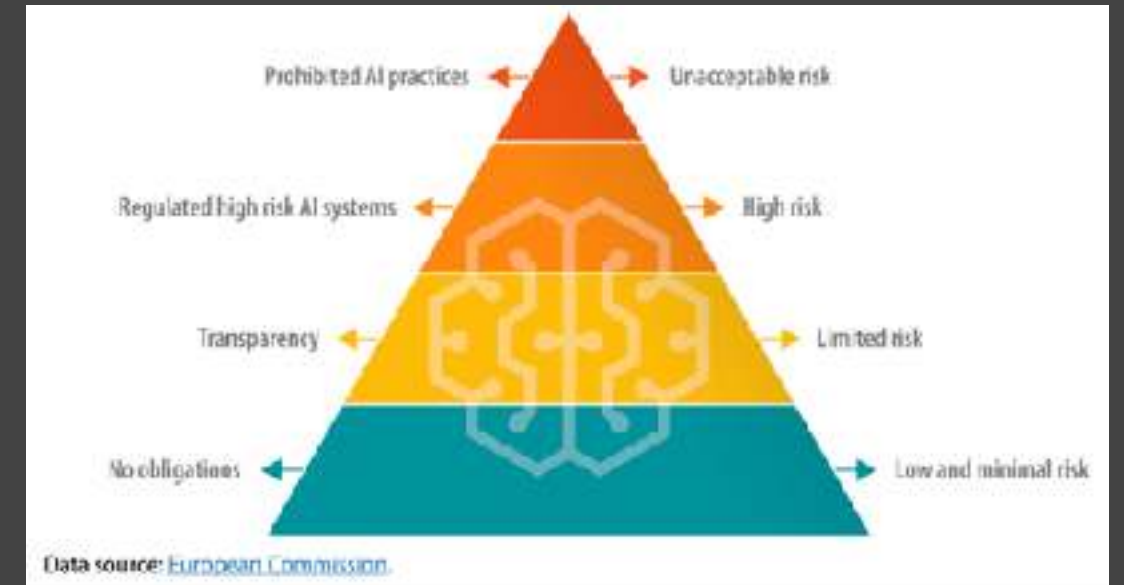


ETHICO-LEGAL GOVERNORS?

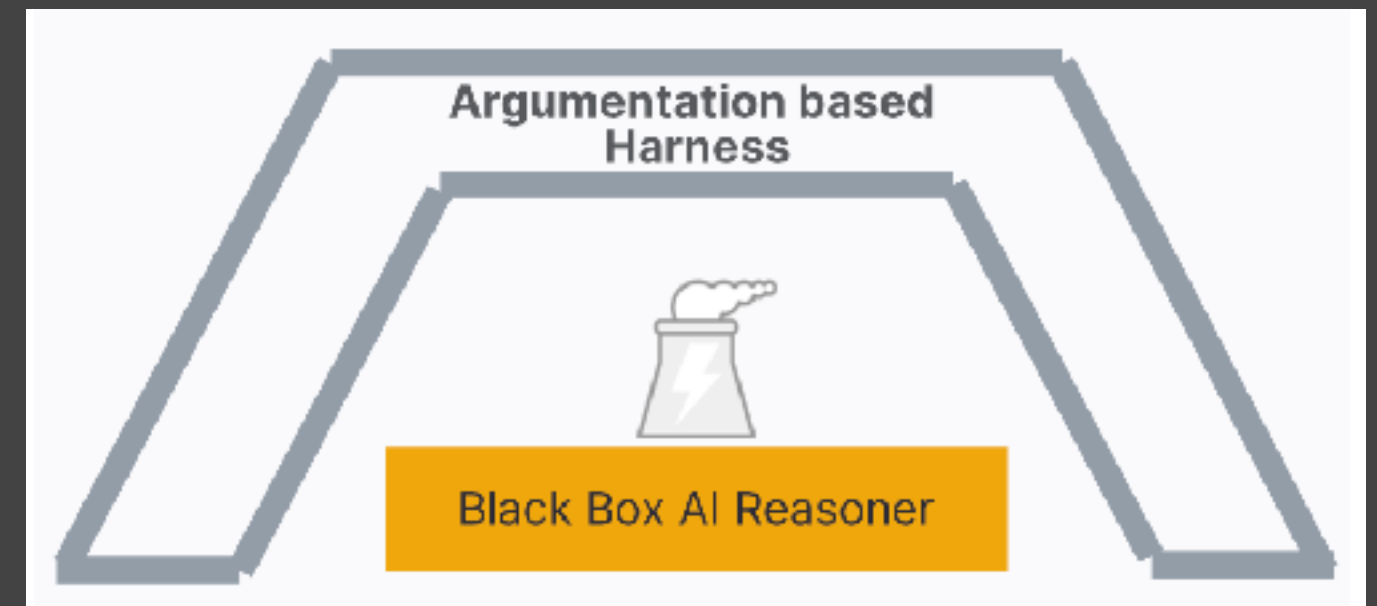
High risk AI applications: 

How to control? How to create trust?

- Opening AI black box systems?
 - Not a good idea (for applications of high criticality)!
 - Limited impact on trust
 - Invitation for adversarial attacks



I argue for control harnesses



Foundational Studies: Ethics

(ongoing PhD studies of David Fuenmayor)



SpringerLink

 Pacific Rim International Conference on Artificial Intelligence
↳ PRICA 2019: **PRICA 2019: Trends in Artificial Intelligence** pp 418–432 | [Cite as](#)

Harnessing Higher-Order (Meta-)Logic to Represent and Reason with Complex Ethical Theories

[David Fuenmayor](#) ✉ & [Christoph Benz Müller](#)

Normative Reasoning with Expressive Logic Combinations

Authors	David Fuenmayor, Christoph Benz Müller
Pages	2903 - 2904
DOI	10.3233/FAIA200445
Category	Research Article
Series	Frontiers in Artificial Intelligence and Applications
Ebook	Volume 325: ECAI 2020

Deontological ethical theory — **PGC by Alan Gewirth**

 **FORMALISATION AND EVALUATION OF ALAN GEWIRTH'S PROOF FOR THE PRINCIPLE OF GENERIC CONSISTENCY IN ISABELLE/HOL**

Home	Title: Formalisation and Evaluation of Alan Gewirth's Proof for the Principle of Generic Consistency in Isabelle/HOL
About	Authors: David Fuenmayor (davfuenmayor@tut.gmail.com) and Christoph Benz Müller
Submission	Submission date: 2018-10-30
Updating Entries	Abstract: An ambitious ethical theory --- Alan Gewirth's "Principle of Generic Consistency" --- is encoded and analysed in Isabelle/HOL. Gewirth's theory has stirred much attention in philosophy and ethics and has been proposed as a potential means to bound the impact of artificial general intelligence.
Using Entries	Change history: [2019-04-09]: added proof for a stronger variant of the PGC and exemplary inferences (revision 88182cb0a2f6)
Search	BibTeX: <code>@article{GewirthPGCProof-AFP, [...]}</code>
Statistics	License: BSD License
Index	
Download	

Our encoding utilises the **dyadic deontic logic** by Carmo and Jones as object logic; in fact, it uses a rich logic combination.

Independent Foundational Studies (using LogiKEY): Ethics

Levanya Singh (Harvard)

Automated Kantian Ethics

No Thumbnail

Citation

Singh, Lavanya. 2022. Automated Kantian Ethics. Bachelor's thesis, Harvard College.

Abstract

AI is beginning to make decisions without human supervision in increasingly consequential contexts like healthcare, polling, and driving. These decisions are inevitably ethically tinged, but most AI systems in use today are not explicitly guided by ethics. Regulators, philosophers, and computer scientists are raising the alarm about the dangers of unethical artificial intelligence, from lethal autonomous weapons to criminal sentencing algorithms prejudiced against people of color. These warnings are spurring interest in automated ethics, or the development of machines that can perform ethical reasoning. Prior work in automated ethics rarely engages with philosophical literature, despite its relevance to the development of responsible AI. If automated ethics draws on philosophical literature, its decisions will be more nuanced, precise, and consistent, but automating ethical theories is difficult in practice. Faithfully translating a complex ethical theory from natural language to the rigid syntax of a computer program is technically and philosophically challenging.

In this thesis, I present an implementation of automated Kantian ethics that is faithful to the Kantian philosophical tradition. Given minimal factual background, my system can judge a potential action as morally obligatory, permissible, or prohibited. To accomplish this, I formalize Kant's categorical imperative, or moral rule, in deontic logic, implement this formalization in the Isabelle/HOL theorem prover, and develop a testing framework to evaluate how well my implementation coheres with expected properties of Kantian ethics, as established in the literature. This testing framework demonstrates that my system outperforms two other potential implementations of automated Kantian ethics. I also use my system to derive philosophically sophisticated and nuanced solutions to two central controversies in Kantian literature: the permissibility of lying (a) in the context of a joke and (b) to a murderer asking about the location of their intended victim. Finally, I examine my system's philosophical implications, demonstrating that it can not only guide AI, but it can also help academic philosophers make philosophical progress and augment the everyday ethical reasoning that we all perform as we navigate the world. Ultimately, I contribute a working proof-of-concept implementation of automated Kantian ethics capable of performing philosophical reasoning more mature than anything previously automated. My work serves as one step towards the development of responsible, trustworthy artificial intelligence.

View/Open

 Automated_Kantian_Ethics_L5_Thesis.pdf
(1.235Mb)

Access Status

Full text of the requested work is not available in DASH at this time ("dark deposit"). For more information on dark deposits, [see our FAQ](#).

Author

Singh, Lavanya

Metadata

[Show full item record](#)

Automating Kantian Ethics in Isabelle A Case Study

Lavanya Singh^[0000-0001-7791-3172]

Harvard University

See her forthcoming papers at

- Isabelle Workshop 2022
- KI 2022

Foundational Studies: Metaphysics

(PhD of Daniel Kirchner)



Principia Logico-Metaphysica (Draft/Excerpt)

Edward N. Zalta
Philosophy Department
Stanford University

With critical theoretical contributions by

Daniel Kirchner
Institut für Mathematik
Freie Universität Berlin
and

Uri Nodelman
Philosophy Department
Stanford University

October 13, 2021

<http://mally.stanford.edu/principia.pdf>

Foundational Studies: Metaphysics

(PhD of Daniel Kirchner)

Principia Logico-Metaphysica (Draft/Excerpt)

Edward N. Zalta
Philosophy Department
Stanford University

With critical theoretical contributions by

Daniel Kirchner
Institut für Mathematik
Freie Universität Berlin
and

Uri Nodelman
Philosophy Department
Stanford University

October 13, 2021

<http://mally.stanford.edu/principia.pdf>



Computer-Verified Foundations of Metaphysics and an Ontology of Natural Numbers in Isabelle/HOL

Dissertation
zur Erlangung des Grades eines
Doktors der Naturwissenschaften

am Fachbereich Mathematik und Informatik
der Freien Universität Berlin

vorgelegt von
Daniel Kirchner

Berlin, December 2021

Entire PhD
thesis was
written
directly in
Isabelle/HOL

Foundational Studies: Metaphysics

(PhD of Daniel Kirchner)

Principia Logico-Metaphysica (Draft/Excerpt)



Edward N. Zalta
Philosophy Department
Stanford University

With critical theoretical contributions by

Daniel Kirchner
Institut für Mathematik
Freie Universität Berlin
and

Uri Nodelman
Philosophy Department
Stanford University

October 13, 2021

<http://mally.stanford.edu/principia.pdf>

Foundational metaphysical theory (based on a hyperintensional relational HO modal logic)

Formalised & studied in Isabelle/HOL

- approx. 24000 loc
- using LogiKEy methodology
- paradox rediscovered & fixed
- derivation of natural numbers



Latest versions of this theory shifted towards **free logic**; strongly influenced (& verified) by **computer-experiments**

Foundational Studies: Metaphysics

(PhD of Daniel Kirchner)



Principia Logico-Metaphysica (Draft/Excerpt)

Edward N. Zalta
Philosophy Department
Stanford University

With critical theoretical contributions by

Daniel Kirchner
Institut für Mathematik
Freie Universität Berlin
and

Uri Nodelman
Philosophy Department
Stanford University

October 13, 2021

<http://mally.stanford.edu/principia.pdf>





THE
REVIEW OF
SYMBOLIC
LOGIC

Edited by
Edward N. Zalta
Daniel Kirchner
Christoph Benz Müller
Uri Nodelman
Daniel Kirchner
Christoph Benz Müller
Edward N. Zalta
Uri Nodelman

MECHANIZING *PRINCIPIA LOGICO-METAPHYSICA* IN FUNCTIONAL TYPE-THEORY

Published online by Cambridge University Press: 12 July 2019

DANIEL KIRCHNER, CHRISTOPH BENZMÜLLER and
EDWARD N. ZALTA

[Show author details](#) ▾



OPEN
PHILOSOPHY

ARTICLE  Open Access

Computer Science and Metaphysics: A Cross-Fertilization

Open Philosophy

Daniel Kirchner, Christoph Benz Müller, Edward N. Zalta August 23, 2019

Isabelle/HOL Code (~24000 loc):

<https://github.com/ekpyron/AOT>

Thesis: <http://dx.doi.org/10.17169/refubium-35141>

Conclusion:

HOL
&
Universal Meta-Logical Reasoning
&
LogiKEY

... it works ...

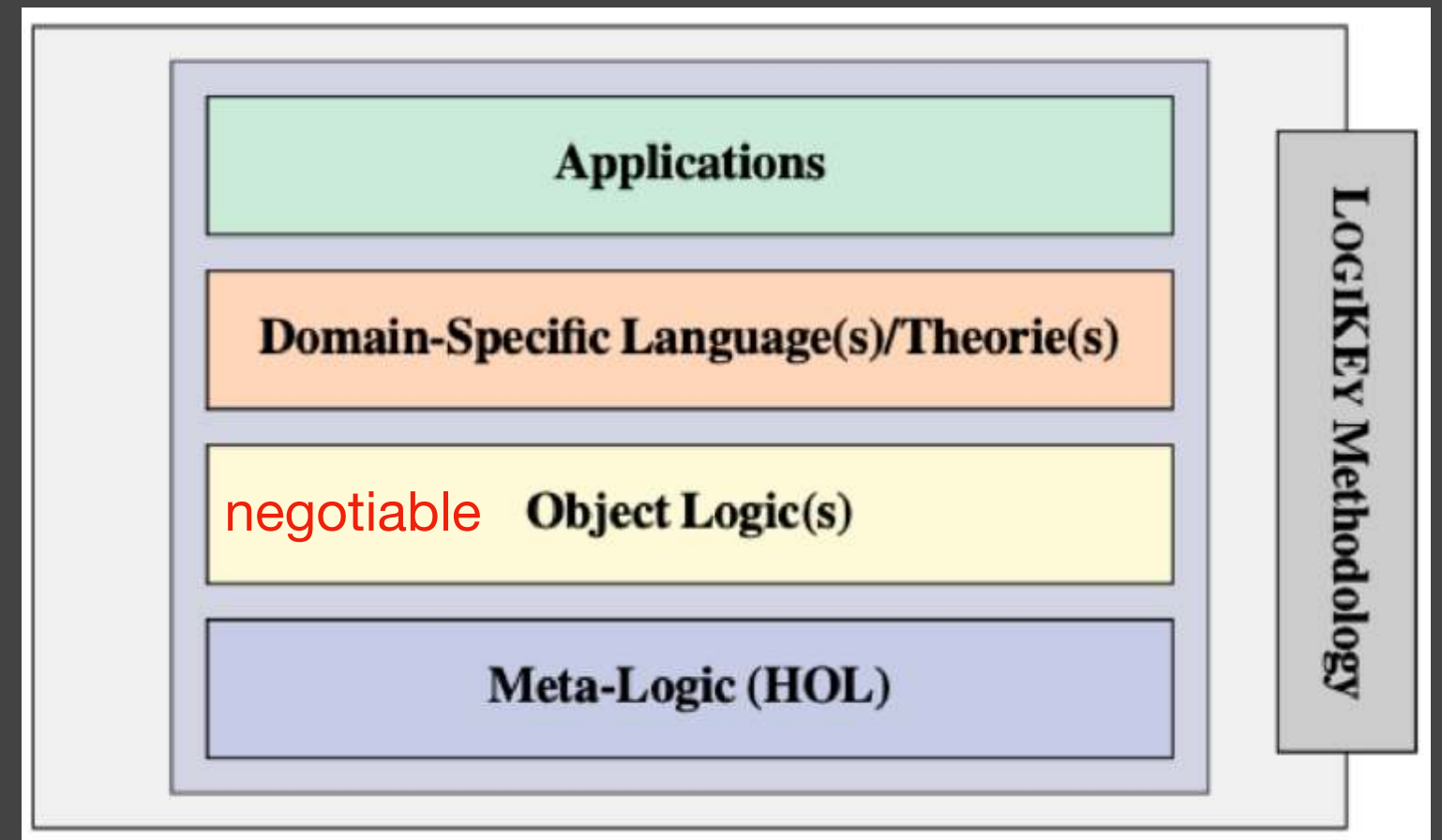
Conclusion: LogiKEy

LogiKEy successfully applied for

- a wide range of object logics
- various object logic combinations
- different application domains (with contribution of new insights)

LogiKEy in Isabelle/HOL

- good proof automation with Sledgehammer
- even more valuable is (counter-)model finding with Nitpick
- very good syntax representations



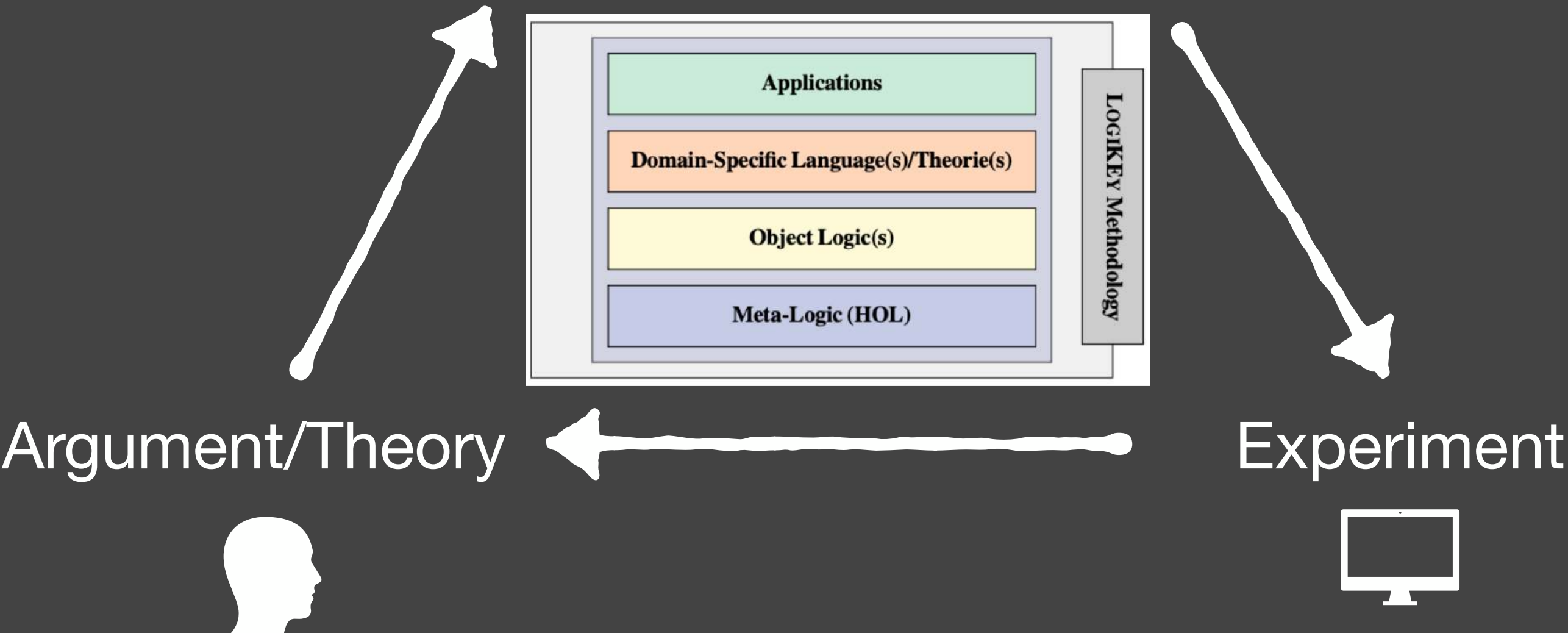
LogiKEy offers a uniform methodology and infrastructure where even object logics and their combinations become negotiable and objects of study.

Conclusion: LogiKEy

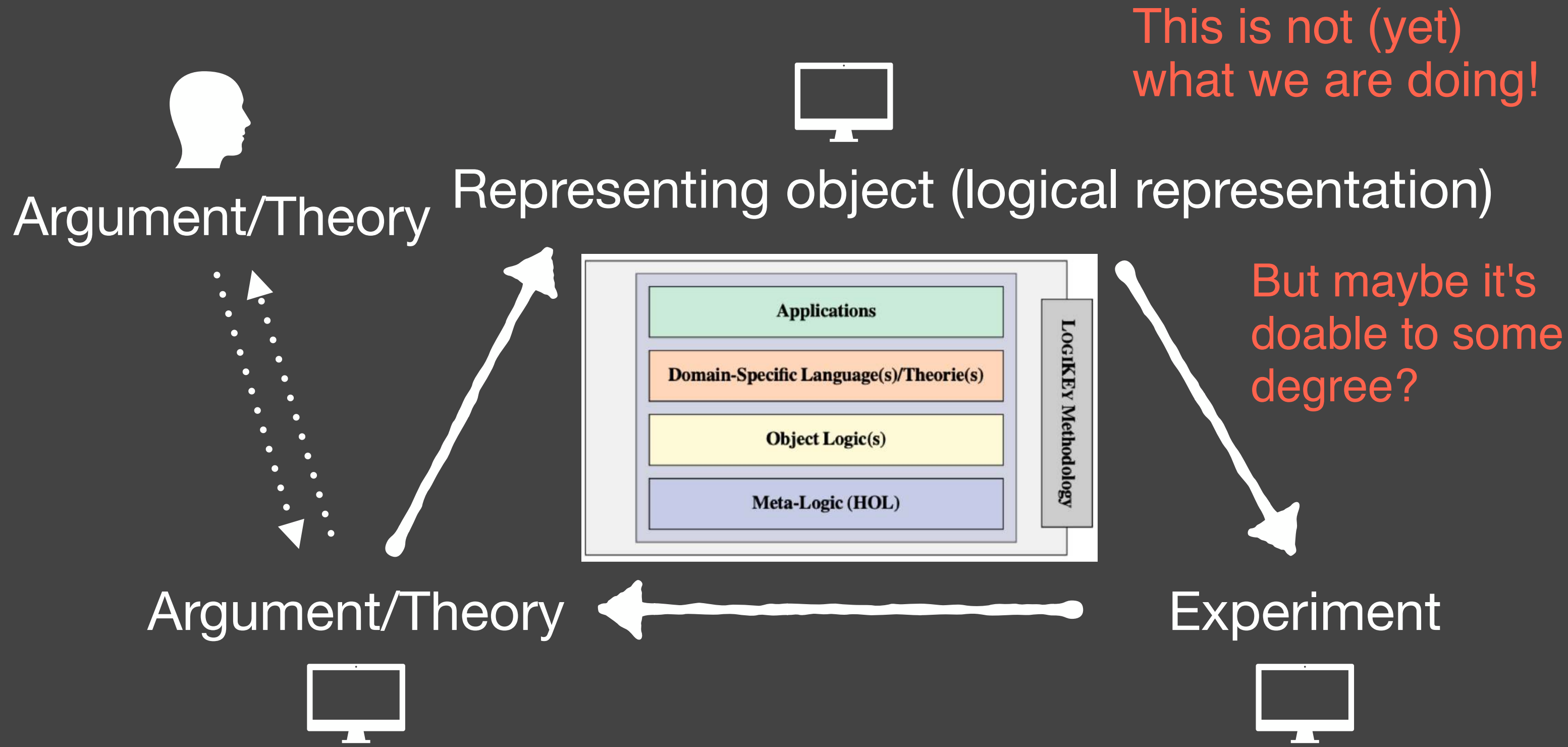
Human-Computer
Interaction



Representing object (logical representation)



Conclusion: LogiKEy



Comment on Maths, Metaphysics & Experimental Sciences

... The difference is that the natural scientists base their answers on observation, experiment, measurement and calculation, while the metaphysicians base theirs on armchair reflection ...

(Timothy Williamson, Oxford, in an article for the British Academy, 14 Aug 2020)

This seems not completely true anymore.

The differences between metaphysics, maths and experimental sciences could gradually disappear?

But clearly: **Representing Objects, Logic & Symbolic AI** are needed.

Reading

(preprints: <http://christoph-benzmueller.de/publications.html>)

HOL (see also the papers of Church, Henkin, Andrews and Muskens in JSL as mentioned before)

- Benzmüller, Brown, Kohlhase (2004), Higher-Order Semantics and Extensionality, J. Symb. Log., <http://doi.org/10.2178/jsl/1102022211>
- ___ & Miller (2014), Automation of Higher-Order Logic, Handbook of the History of Logic, <http://doi.org/10.1016/B978-0-444-51624-4.50005-8>
- ___ & Andrews (2019), Church's Type Theory, SEP, <https://plato.stanford.edu/entries/type-theory-church/>

Leo Provers

- Steen & Benzmüller (2021), Extensional Higher-Order Paramodulation in Leo-III, J. Autom. Reason., <http://doi.org/10.1007/s10817-021-09588-x>
- Benzmüller, Sultana, Paulson (2015), Thhe Higher-Order Prover Leo-II, J. Autom. Reason., <http://doi.org/10.1007/s10817-015-9348-y>
- ___ & Kohlhase (1998), LEO – A Higher-Order Theorem Prover, CADE, <http://doi.org/10.1007/BFb0054256>

E Prover

- See the references at: <https://www.lehre.dhbw-stuttgart.de/~sschulz/E/References.html>

LogiKEy & Universal (Meta-)Logical Reasoning

- Benzmüller (2019), **Universal (meta-)logical reasoning: Recent successes**. Sci. Comp. Progr. (<http://doi.org/10.1016/j.scico.2018.10.008>)
- ___, Parent & van der Torre (2020), **Designing normative theories for ethical and legal reasoning: LogiKEy framework, methodology, and tool support**. Artificial Intelligence. (<https://doi.org/10.1016/j.artint.2020.103348>)
- Benzmüller et.al. (2020), **LogiKEy Workbench: Deontic Logics, Logic Combinations and Expressive Ethical and Legal Reasoning (Isabelle/HOL Dataset)**. Data in Brief (<https://doi.org/10.1016/j.dib.2020.106409>)
- Bibel (2022), **Computer Kreiert Wissenschaft**. Informatik Spektrum. (<https://doi.org/10.1007/s00287-022-01456-1>)

Reading (cont'd)

(preprints: <http://christoph-benzmueller.de/publications.html>)

Foundational Studies in Metaphysics

- Kirchner, Benz Müller & Zalta (2019), **Computer Science and Metaphysics: A Cross-Fertilization**. Open Philosophy (<http://doi.org/10.1515/opphil-2019-0015>)
- __ (2020), **Mechanizing Principia Logico-Metaphysica in Functional Type Theory**. Review of Symbolic Logic (<https://doi.org/10.1017/S1755020319000297>)
- Kirchner (2021). **Embedding of Abstract Object Theory in Isabelle/HOL**. Full sources. (<https://github.com/ekpyron/AOT/tree/dissertation>)
- __ (2022), **Computer-Verified Foundations of Metaphysics and an Ontology of Natural Numbers in Isabelle/HOL**. PhD thesis, Dep. of Maths and CS, FU Berlin.

Foundational Studies in Ethics & Law

- Fuenmayor & Benz Müller (2018), **Formalisation and Evaluation of Alan Gewirth's Proof for the Principle of Generic Consistency in Isabelle/HOL**. (<https://www.isa-afp.org/entries/GewirthPGCProof.html>)
- __ (2019), **Harnessing Higher-Order (Meta-)Logic to Represent and Reason with Complex Ethical Theories**. PRICAI 2019 (https://doi.org/10.1007/978-3-030-29908-8_34)
- Benz Müller & Fuenmayor (2021), **Value-oriented Legal Argumentation in Isabelle/HOL**. ITP 2021 (<https://doi.org/10.4230/LIPIcs.ITP.2021.7>)
- __, Fuenmayor & Lomfeld (2022), **Modelling Value-oriented Legal Reasoning in LogiKEY**. (<https://arxiv.org/abs/2006.12789>)

Studies on the Ontological Argument

- Benz Müller & Woltzenlogel Paleo (2014), **Automating Gödel's Ontological Proof of God's Existence with Higher-order Automated Theorem Provers**. ECAI 2014 (<http://doi.org/10.3233/978-1-61499-419-0-93>)
- __ (2016), **The Inconsistency in Gödel's Ontological Argument: A Success Story for AI in Metaphysics**. IJCAI 2016 (<http://www.ijcai.org/Proceedings/16/Papers/137.pdf>)

Reading (cont'd)

(preprints: <http://christoph-benzmueller.de/publications.html>)

Studies on the Ontological Argument (cont'd)

- Benz Müller & Woltzenlogel Paleo (2016), **An Object-Logic Explanation for the Inconsistency in Gödel's Ontological Theory** (Extended Abstract, Sister Conferences). KI 2016 (<http://doi.org/10.1007/978-3-319-46073-4>)
- __ (2017). **Computer-Assisted Analysis of the Anderson-Hájek Controversy**. Logica Universalis (<http://doi.org/10.1007/s11787-017-0160-9>)
- Fuenmayor & Benz Müller (2017), **Automating Emendations of the Ontological Argument in Intensional Higher-Order Modal Logic**. KI 2017 (http://doi.org/10.1007/978-3-319-67190-1_9)
- __ (2018). **A Case Study On Computational Hermeneutics: E. J. Lowe's Modal Ontological Argument**. IfCoLoG Journal of Logics and their Applications (<https://www.researchgate.net/publication/333804824>)
- Benz Müller & Fuenmayor (2020), **Computer-supported Analysis of Positive Properties, Ultrafilters and Modal Collapse in Variants of Gödel's Ontological Argument**. Bulletin of the Section of Logic (<http://doi.org/10.18778/0138-0680.2020.08>)
- Benz Müller (2020), **A (Simplified) Supreme Being Necessarily Exists, says the Computer: Computationally Explored Variants of Gödel's Ontological Argument**. KR 2020 (<https://doi.org/10.24963/kr.2020/80>)
- __ (2022), **A Simplified Variant of Gödel's Ontological Argument**. To appear (<https://www.researchgate.net/publication/358607847>)

Isabelle/HOL:

- Website: <https://isabelle.in.tum.de>
- Documentation: <https://isabelle.in.tum.de/documentation.html>
- Nipkow, Paulson & Wenzel (2002), **Isabelle/HOL: A Proof Assistant for Higher-Order Logic**. Springer (<https://doi.org/10.1007/3-540-45949-9>)
- Blanchette, Bulwahn & Tobias Nipkow (2011), **Automatic Proof and Disproof in Isabelle/HOL**. FroCoS 2011 (https://doi.org/10.1007/978-3-642-24364-6_2)

Reading

(preprints: <http://christoph-benzmueller.de/publications.html>)

Free Logic & Axiomatic Category Theory in Free Logic

- Scott (1967), **Existence and description in formal logic**. Reprinted in: Lambert (1991), Philosophical Application of Free Logic, OUP. (ISBN 0195061314)
- Scott (1977), **Identity and existence in intuitionistic logic**. Applications of Sheaves, Springer (<https://doi.org/10.1007/BFb0061839>)
- Benz Müller & Scott (2016), **Automating Free Logic in Isabelle/HOL**. ICMS 2016 (https://doi.org/10.1007/978-3-319-42432-3_6)
- __ (2018), **Axiom systems for category theory in free logic**. Archive of Formal Proofs (<https://www.isa-afp.org/entries/AxiomaticCategoryTheory.html>)
- __ (2020), **Automating Free Logic in HOL, with an Experimental Application in Category Theory**. JAR (<http://doi.org/10.1007/s10817-018-09507-7>)
- Tiemens, Scott, Benz Müller & Benda (2020), **Computer-supported Exploration of a Categorical Axiomatization of Modeloids**. RAMiCS 2020 (https://doi.org/10.1007/978-3-030-43520-2_19)
- Makarenko & Benz Müller (2020), **Positive Free Higher-Order Logic and its Automation via a Semantical Embedding**. KI 2020 (http://doi.org/10.1007/978-3-030-58285-2_9)
- Bayer (2021), **Exploring categories, formally**. BSc Thesis, Dep. of Maths and CS, FU Berlin.
- Gonus (2021), **Categorical semantics of Intuitionistic Multiplicative Linear Logic and its formalization in Isabelle/HOL**. MSc Thesis, Dep. of Maths and CS, FU Berlin.